audita

SMART CONTRACT SECURITY REVIEW · PREPARED FOR

OLY Stock Vault

Robinhood Chain stock vault and revenue flows

DATE	COMMIT	REMEDIATION
October 15, 2026	5a451ab	49c9305

01 ABOUT

Audita

Audita is a Web3 security company specializing in smart contract security reviews for protocols, decentralized applications, and on-chain infrastructure.

We help teams identify vulnerabilities before they reach production through rigorous manual code review, adversarial security research, architecture analysis, and automated testing.

Our team consists of 50+ senior security researchers with proven experience across DeFi, infrastructure, wallets, trading systems, and complex on-chain applications.

To date, Audita has completed more than 100 security reviews across the Web3 ecosystem, working with teams at different stages ranging from pre-launch protocols to production systems securing significant on-chain value.

Every engagement is approached from both an offensive and defensive perspective: understanding how the system is intended to behave, identifying where those assumptions can break, and determining how an attacker could exploit them.

PREVIOUS WORK	WEBSITE
Audita public reports	audita.io

02 METHODOLOGY

How this review works

Researchers inspect the agreed source scope and audited commit, trace asset flows and privileged actions, challenge trust assumptions, and reproduce credible attack paths with targeted tests where feasible.

Each accepted finding records its impact and recommended remediation. We then review the remediation commit and record the outcome as Fixed, Partly fixed, or Acknowledged for that finding.

Risk classification

Severity reflects both the impact of a successful exploit and the likelihood of reaching it under the stated assumptions.

Likelihood / Impact	High	Medium	Low
High	HIGH	HIGH	MEDIUM
Medium	HIGH	MEDIUM	LOW
Low	MEDIUM	LOW	LOW

Informational findings have no direct security impact and sit outside the matrix.

IMPACT

High	Major asset loss, critical control compromise, or protocol-wide disruption.
Medium	Material but bounded loss or disruption.
Low	Limited loss, disruption, or scope.

LIKELIHOOD

High	Practical under ordinary on-chain conditions.
Medium	Requires specific but plausible conditions.
Low	Requires unusual conditions or disproportionate resources.

DISCLAIMER

A security review is bounded by time, resources, expertise, and the stated scope. It can identify vulnerabilities but cannot guarantee their absence. Resolution statuses describe only the remediation commit reviewed here. Later code, configuration, or integrations may change the risk. Continued testing, monitoring, and independent review are recommended.

03 EXECUTIVE SUMMARY

FINDINGS	FIXED & VERIFIED	PARTIAL OR ACKNOWLEDGED
50	43	7

SEVERITY DISTRIBUTION

3 High	11 Medium	26 Low	10 Informational

CONTRACTS IN SCOPE

src/StockRelay.sol	src/robinhood/OLYTaxHook.sol
src/robinhood/RHBuyNBurn.sol	src/robinhood/RHOracleSwapActions.sol
src/robinhood/RHOracleSwapLib.sol	src/robinhood/farm/RHFarmKeeper.sol
src/robinhood/farm/libs/RHFarmKeeperLib.sol	src/robinhood/farm/libs/RHFarmKeeperUniLib.sol
src/robinhood/farm/libs/RHFarmKeeperV3Lib.sol	src/robinhood/farm/libs/RHFarmKeeperV4Lib.sol
src/robinhood/farm/libs/RHFarmKeeperViewLib.sol	src/robinhood/farm/libs/RHFarms.sol
src/robinhood/interfaces/IPriceProvider.sol	src/robinhood/interfaces/IRHFarmKeeper.sol
src/robinhood/oracles/V3TwapPriceProvider.sol	src/robinhood/oracles/V4TwapPriceProvider.sol
src/robinhood/stock/RHTaxRouter.sol	src/robinhood/stock/StockBuyer.sol
src/robinhood/stock/StockPayouts.sol	

TIMELINE · OCTOBER 2026

Audit · 9 days		Remediation · 4 days	
KICK-OFF	END OF AUDIT	REMEDIATIONS START	REMEDIATIONS END
October 2, 2026	October 10, 2026	October 11, 2026	October 14, 2026

STARTING COMMIT HASH	5a451abf4867bb22ff28e2c8c5046ab2aeec8aa6
FINAL COMMIT HASH	49c930556d18cf021775d492cdb9c195b44c922f

04 PROJECT SUMMARY

What OLY Stock Vault is

Ethereum L1 users stake OLY for pro-rata Robinhood Chain payouts of tokenized stocks, farm fees, and ETH.

An OLY swap tax funds stock purchases and farm operations. Bought stocks and harvested fees reach an ownerless 28-day payout ledger that uses relayed L1 snapshot weights to pay stakers. A separate burner buys OLY and initiates its L1 burn.

SECURITY POSTURE

BEFORE REMEDIATION 5a451ab	AFTER REMEDIATION 49c9305

Revenue and payout boundaries

M-08 · L-01 · L-13 · I-06

Relay, ledger, and router accounting could misassign cycle revenue, strand value, or alter credited entitlements.

Oracle and swap execution

H-02 · H-03 · M-09 · M-10

Oracle history and swap checks could block quotes or price stock purchases, farm swaps, and burns against the wrong reference.

Farm liquidity and harvest lifecycle

H-01 · L-23 · L-25 · L-26

Farm ranges, caps, hook behavior, and batch failures could block deposits and harvests or leave principal idle.

Stock purchase rounds and incentives

M-01 · M-03 · M-04 · M-11

Shared rounds, USDG balances, and bounty recalculation could skip stocks, exceed caps, or overpay callers.

43 of 50 findings fixed and verified. The final commit addresses farm ranges, stock buying, oracle reads, swap handling, the OLY burn path, and payouts.

One payout-shortfall finding is Partly fixed, with total-wipe, reversed-burn, and direct-transfer cases remaining. Six findings are Acknowledged.

05 CONTRACTS & ROLES

Core contracts

CONTRACT	ROLE
`src/StockRelay.sol`	Relays L1 staking weights and payout destinations to the L2 ledger.
`src/robinhood/OLYTaxHook.sol`	Taxes OLY swaps in ETH and records pool oracle observations.
`src/robinhood/RHBuyNBurn.sol`	Buys OLY with ETH and initiates its L1 burn.
`src/robinhood/farm/RHFarmKeeper.sol`	Manages farm liquidity and sends harvest revenue to payouts.
`src/robinhood/oracles/V3TwapPriceProvider.sol`	Quotes pairs from Uniswap v3 TWAP observations.
`src/robinhood/oracles/V4TwapPriceProvider.sol`	Quotes pairs from Uniswap v4 TWAP observations.
`src/robinhood/stock/RHTaxRouter.sol`	Splits ETH tax and permits owner-authorized upgrades.
`src/robinhood/stock/StockBuyer.sol`	Buys tokenized stocks with tax ETH and forwards them to payouts.
`src/robinhood/stock/StockPayouts.sol`	Books 28-day revenue and pays stakers using relayed L1 weights.

PRIVILEGED ROLES

Farm keeper owner. The initialized owner enables farms, sets routes and slippage, and upgrades the keeper.

Tax router owner. The initialized owner sets tax destinations and genesis allocation, and upgrades the router.

Stock buyer and burner owners. Deployment owners set assets, venues, feeds, caps, incentives, and pause state.

Price provider owners. Deployment owners register and remove TWAP routes.

L1 relay alias. The immutable aliased StockRelay address credits payout cycle weights.

06 FINDINGS

What we found

ID	TITLE	SEVERITY	STATUS
H-01	Anyone can block every RHFarmKeeper liquidity add by filling a full-range end tick	HIGH	FIXED
H-02	The swap ladder cannot tell a slippage failure from a caller-held venue lock, and StockBuyer still charges the interval, so any caller can turn every `buy` into a no-op that resets the cooldown	HIGH	FIXED
H-03	The burner calls a function that bridged OLY does not support	HIGH	FIXED
M-01	`StockBuyer`'s bounty ramp counts blocks in which nobody could buy, so the first caller after a pause gets the maximum bounty	MEDIUM	FIXED
M-02	Exact-out swaps are taxed on the net ETH leg instead of the gross, so they pay about 4.76% instead of the fixed 5%	MEDIUM	FIXED
M-03	Leftover USDG ignores `buyCap` and can be swapped all at once	MEDIUM	FIXED
M-04	One shared cooldown lets a caller keep chosen stocks out of every `buy`	MEDIUM	FIXED
M-05	The permissionless `buy` caller can sandwich the protocol's own swaps inside a single transaction and capture most of the oracle slippage band on low-fee venues	MEDIUM	FIXED
M-06	Partial fills still pay tax on the full requested amount	MEDIUM	FIXED
M-07	Off-chain oracle pause blocks TWAP quotes for stock tokens	MEDIUM	FIXED
M-08	Stakers who end a stake just after a cycle's snapshot still collect that whole cycle's payout	MEDIUM	ACKNOWLEDGED

06 FINDINGS

CONTINUED

ID	TITLE	SEVERITY	STATUS
M-09	Free swaps before the OLY pool holds liquidity fill the oracle ring and remove the burner's slippage floor	MEDIUM	FIXED
M-10	A lagging TWAP lets a caller sandwich a protocol swap at the pre-move price after a real move	MEDIUM	FIXED
M-11	Changing the buy interval or the incentive band re-prices the time already elapsed, so shortening the interval jumps the next bounty to `maxIncentiveBps`	MEDIUM	FIXED
L-01	Dead shares inflate `totalWeight` and strand part of every cycle's pot	LOW	FIXED
L-02	Revenue accrued to a cycle with no stakers can never be paid out	LOW	FIXED
L-03	The TWAP providers return stale prices after quiet periods or sequencer halts, which breaks the `IPriceProvider` "MUST revert" rule	LOW	FIXED
L-04	The default farm swap slippage (5%) is below the taxed OLY/ETH pool's own 6% cost, so an OLY farm can never rebalance	LOW	FIXED
L-05	TWAP routes are wired to pools whose observation rings are smaller than the 1-hour window, so an griefer can make price reads revert	LOW	FIXED
L-06	`claimable` and `previewPayout` overstate payouts during a shortfall	LOW	FIXED
L-07	The `StockBuyer` owner can drain staker ETH through a fake price provider, although the NatSpec promises there is no owner extraction path	LOW	ACKNOWLEDGED

06 FINDINGS

CONTINUED

ID	TITLE	SEVERITY	STATUS
L-08	Native-in v4 swaps fail when another currency is left synced in the PoolManager	LOW	FIXED
L-09	Anyone can trigger the keeper's protocol-owned swaps and sandwich them in the same transaction, because the only protection is a TWAP-relative floor with no spot check	LOW	FIXED
L-10	Farm deposits revert once the farm's own swap moves the pool price by more than `liquiditySlippage`	LOW	FIXED
L-11	A staker can make harvest syncs fail from inside `claim` and push that revenue into the next cycle	LOW	FIXED
L-12	An outside balance drop breaks the keeper's shared books	LOW	FIXED
L-13	A haircut claim is booked as fully paid, and later revenue silently fills the hole	LOW	PARTLY
L-14	The V3 ETH-swap path picks the WETH side of the intermediary pool without checking that exactly one side is flagged	LOW	FIXED
L-15	Stock tokens sent directly to `StockBuyer` are permanently stuck	LOW	FIXED
L-16	Relay overpayments remain at an inaccessible L2 address	LOW	FIXED
L-17	Quiet pools can lose working TWAP quotes	LOW	FIXED
L-18	Skipped oracle updates misweight price changes	LOW	FIXED
L-19	Rebalancing swap on v4 farms ignores the protocol fee and swaps too little	LOW	FIXED

06 FINDINGS

CONTINUED

ID	TITLE	SEVERITY	STATUS
L-20	Buy-and-burn floor ignores the 5% OLY tax, leaving about 2% of its 8% slippage tolerance	LOW	FIXED
L-21	The 28-day harvest gate runs on a wall clock unrelated to the payout ledger's L1-block cycles, so each farm's fees are booked up to a full cycle late	LOW	ACKNOWLEDGED
L-22	The V3 add-liquidity bounds price exactly 1e18 raw units of token0 as an integer, so for cheap 18-decimal token0 pairs the bounds are coarse or divide by zero and deposits revert	LOW	ACKNOWLEDGED
L-23	Fixed liquidity cut leaves part of every V4 deposit unused, and the raised cap does not undo it	LOW	FIXED
L-24	`validateV4Pool` accepts any hook, so a V4 farm pool's hook can keep up to `liquiditySlippage` of the farm's principal on adds and on removal, or lock the position by reverting on liquidity removal	LOW	ACKNOWLEDGED
L-25	A liquidity add sized from a lagging TWAP lets a caller mint the farm's position at the pre-move price	LOW	FIXED
L-26	One farm that cannot harvest blocks `harvestAllFarms` for every farm	LOW	FIXED
I-01	`quoteSubmissionFee` comment names a stale field and the quote has no basefee margin	INFORMATIONAL	FIXED
I-02	Removing the last USDG-quoted stock strands the leftover USDG	INFORMATIONAL	FIXED
I-03	`RHTaxRouter` does not disable initializers on its implementation, so anyone can initialize it and become its owner	INFORMATIONAL	FIXED

06 FINDINGS

CONTINUED

ID	TITLE	SEVERITY	STATUS
I-04	Nested swaps clear the swap flag while the outer swap is still running	INFORMATIONAL	FIXED
I-05	`setAllocPoint` accepts 0 although enabling a farm requires at least 50	INFORMATIONAL	ACKNOWLEDGED
I-06	`setDestinations` applies a new split to tax that has already accrued, so the owner can redirect the whole undistributed balance	INFORMATIONAL	FIXED
I-07	Removing a V3 farm leaves its empty position NFT unburned in the keeper	INFORMATIONAL	FIXED
I-08	V4 amount caps round down while the pool rounds the cost up, so deposits revert when the slippage raise rounds to zero	INFORMATIONAL	FIXED
I-09	Intermediary pairs that trade only in hookless v4 pools are unpriceable, so farms cannot route ETH through them	INFORMATIONAL	FIXED
I-10	A pause guard that returns any value other than 0 or 1 makes every quote on the route revert, instead of the documented "not paused"	INFORMATIONAL	FIXED

HIGH SEVERITY · 3 FINDINGS

H-01 HIGH FIXED

Anyone can block every RHFarmKeeper liquidity add by filling a full-range end tick

AFFECTED CODE

- `src/robinhood/farm/libs/RHFarmKeeperV4Lib.sol`, _prepareIncreaseUnlockDate()
- `src/robinhood/farm/libs/RHFarmKeeperV3Lib.sol`, addLiquidity()
- `src/robinhood/farm/RHFarmKeeper.sol`, depositToFarm()

DESCRIPTION

`RHFarmKeeper` holds one full-range position per farm. Both the V4 library and the V3 library hard-code its ticks to the usable extremes of the pool:

```
int24 tickLower = TickMath.minUsableTick(farm.poolKey.tickSpacing);
int24 tickUpper = TickMath.maxUsableTick(farm.poolKey.tickSpacing);
```

Every full-range position in a pool uses these same two ticks, and any account can open a position on one of them.

Uniswap caps `liquidityGross`, the summed liquidity of all positions that use a tick as a bound, at `maxLiquidityPerTick`. The cap is `type(uint128).max` divided by the number of usable ticks (V4, V3). Both versions enforce it on every add. In V4, `Pool.modifyLiquidity` reverts with `TickLiquidityOverflow`:

```
if (liquidityDelta >= 0) {
    uint128 maxLiquidityPerTick = tickSpacingToMaxLiquidityPerTick(params.tickSpacing);
    if (state.liquidityGrossAfterLower > maxLiquidityPerTick) {
        TickLiquidityOverflow.selector.revertWith(tickLower);
    }
    if (state.liquidityGrossAfterUpper > maxLiquidityPerTick) {
        TickLiquidityOverflow.selector.revertWith(tickUpper);
    }
}
```

In V3, `Tick.update` reverts with `LO`. The check runs only when liquidity is added. Fee collection passes a zero delta and skips it.

An attacker opens a position one tick spacing wide that ends on a farm tick and fills that tick to the cap. Every later add on that tick, however small, reverts. The range lies wholly above or below the current price, so it takes a single token, and the square root of the price at the end ticks is about `2^64` (top) or `2^-64` (bottom). With tick spacing `s`, the cost is:

$$\Delta x \approx \frac{L_{max}}{\sqrt{P(t_{max})}}\left(1.0001^{s/2}-1\right), \qquad \Delta y \approx L_{max}\sqrt{P(t_{min})}\ \left(1.0001^{s/2}-1\right)$$

The OLY/ETH pool uses tick spacing `200`, so `maxLiquidityPerTick` is about `3.83e34` and filling either end tick costs about `2.1e13` wei: ETH for `maxUsableTick`, OLY for `minUsableTick`. A V4 farm with tick spacing `100` costs about `5.2e12` wei, and a V3 farm with tick spacing `10` about `5.2e10` wei.

Nothing stops third parties from using the farm ticks. `OLYTaxHook` does not enable `beforeAddLiquidity`, V3 pools have no hooks, and `enableV4Farm` accepts pools with any hook. `RHFarmKeeper` has no setter for a farm's ticks.

IMPACT

Once a farm tick is full, `depositToFarm` reverts for that farm. It is the only path by which farm balances become liquidity, and the block lasts until the attacker closes the position.

- Protocol revenue meant for liquidity sits idle. `OLYTaxHook` takes a 5% ETH tax on every OLY swap and sends it to `RHTaxRouter`, which the deploy script configures to route 68% of it to `RHFarmKeeper`. `receive` credits that ETH to farm balances that can no longer be deposited. The OLY/ETH pool stays thinner than intended, so OLY trades move the price more.
- Fees stop compounding. `harvestFarm` credits the 32% of harvested fees meant for reinvestment to the same idle balances, so the position stops growing.
- A farm filled before its first deposit never gets a position, since the first mint reverts too.

The attacker needs no role, pays only gas, recovers the dust on exit, and gains nothing. The owner has no good response. `removeFarm` still works, but for the OLY farm it pulls all protocol liquidity out of the main OLY pool, and re-enabling the farm reuses the same ticks. Only an upgrade of `RHFarmKeeper` fixes the issue. Inflows, fee collection, harvests, and `removeFarm` keep working. No funds leave protocol control.

RECOMMENDATION

Place farm positions well inside the usable range instead of on `minUsableTick` and `maxUsableTick`. The cost to fill a tick grows fast as the tick moves inward. With tick spacing `200`, a one-spacing range ending at tick `887,200` takes `0.00002` whole tokens, at `600,200` about `36`, and at `350,200` about `9.6` million. Each end is paid in a different token (the top in token0, the bottom in token1), so make the range a per-farm parameter chosen with the value of each pool token in mind. The attacker recovers these tokens on exit, so the deterrent is the capital locked, not money lost.

`OLYTaxHook` cannot reject third-party positions on the farm ticks: it does not enable `beforeAddLiquidity` and is immutable and ownerless. The range change is therefore the only fix for the OLY/ETH farm.

H-02 HIGH FIXED

The swap ladder cannot tell a slippage failure from a caller-held venue lock, and StockBuyer still charges the interval, so any caller can turn every `buy` into a no-op that resets the cooldown

AFFECTED CODE

- `src/robinhood/RHOracleSwapActions.sol` , _swapExactIn()
- `src/robinhood/stock/StockBuyer.sol` , buy()

DESCRIPTION

The ladder's catch swallows every revert of `this.executeSwap` , including lock reverts the caller creates on purpose: UniswapV3Pool `LOK` while the caller is inside the pool's `flash` callback, or PoolManager `AlreadyUnlocked` while the caller holds the PM unlock (the pm.unlock call reverts before `unlockCallback` is reached). Meanwhile `buy` writes lastBuyBlock before the legs run and keeps it when `filledWad == 0` . The comment accepts that a failed round consumes the interval, but only considers failures nobody controls, not a caller who manufactures the failure.

IMPACT

By calling first after each `buyInterval` (about one hour) from inside a held lock, an attacker blocks StockBuyer's ETH-to-stock conversion for as long as it pays gas. The ETH is not stolen, but it stays unconverted, the incentive ramp never builds for honest keepers, and the backlog later drains at only `buyCap` per interval, so stakers of the attacked cycles lose their share to later cohorts. Both V3 and V4 venues have a lock the caller can hold. This is pure griefing.

RECOMMENDATION

Only advance `lastBuyBlock` when `filledWad != 0` , or revert when nothing was bought. In `_swapExactIn` , catch only `RHOracleSwapActions__TooLittleReceived` and rethrow every other error, and revert at the start of `buy` when the PoolManager is unlocked.

H-03 HIGH FIXED

The burner calls a function that bridged OLY does not support

AFFECTED CODE

- `src/robinhood/RHBuyNBurn.sol`, buyNBurn()
- `script/robinhood/DeployRobinhoodCore.s.sol`, OLY_RH, _newBuyNBurn()

DESCRIPTION

`RHBuyNBurn` calls `oly.burn(amount)` after every purchase. The canonical bridged OLY token configured for deployment does not support this function. The call therefore reverts and rolls back the entire purchase. This incompatibility was confirmed from the deployed token's bytecode.

IMPACT

With the planned deployment, roughly 30% of tax revenue would collect in a burner that cannot complete a purchase. The burner cannot withdraw ETH, change its OLY token or be upgraded, so its owner cannot recover those funds through the contract. Redirecting future tax payments does not recover ETH already received.

RECOMMENDATION

Use a disposal method supported by the bridged token and verify it before funding the burner. If the goal is to reduce L1 supply, use an appropriate bridge and L1 burn flow. Sending tokens to a dead address does not reduce `totalSupply`.

MEDIUM SEVERITY · 11 FINDINGS

M-01 MEDIUM FIXED

`StockBuyer`'s bounty ramp counts blocks in which nobody could buy, so the first caller after a pause gets the maximum bounty

AFFECTED CODE

- `src/robinhood/stock/StockBuyer.sol`, currentIncentiveBps(), buy()

DESCRIPTION

`currentIncentiveBps` ramps up based on blocks since lastBuyBlock, which only `buy()` updates. Blocks during a pause, while the balance is empty or the basket is empty, or during a sequencer halt all count as "nobody called". On Arbitrum Orbit `block.number` is an L1 estimate that jumps after a halt. The first caller afterwards is paid `maxIncentiveBps` (up to 10%) instead of the base rate.

IMPACT

In each such episode, `(max - base) × budget` of staker ETH goes to whoever calls first, even though no keeper work was delayed.

RECOMMENDATION

Reset `lastBuyBlock` on unpause, when the first funds arrive, and when an asset is added, or measure elapsed time from the moment `buy` first became callable.

M-02 MEDIUM FIXED

Exact-out swaps are taxed on the net ETH leg instead of the gross, so they pay about 4.76% instead of the fixed 5%

AFFECTED CODE

- `src/robinhood/OLYTaxHook.sol`, _beforeSwap(), _afterSwap()

DESCRIPTION

Both branches compute `fee = amt * FEE_BPS / BPS` (L101-L102, L121-L123), but on different bases. For exact-in swaps `amt` is the gross ETH leg. For exact-out swaps it is the net leg: the ETH the seller receives on an exact-out sell (the pool then pays out `amt + fee`), and the pool's fee-exclusive input on an exact-out buy (the buyer pays `amt + fee`). The exact-out base is never grossed up, so those trades pay 500 / 10_500 = 4.762% of the gross ETH leg instead of the fixed 5% promised in the NatSpec.

IMPACT

Any trader or aggregator can route all OLY flow as exact-out and pay about 0.238% less of the ETH volume, roughly 4.8% of the tax on that flow. For example, an exact-out sell for 0.95 ETH pays 0.0475 ETH of tax on a 0.9975 ETH gross leg, against 0.05 ETH for an exact-in sell of 1 ETH. The shortfall is lost by `RHTaxRouter` and its recipients (the genesis wallet, the farm keeper and therefore stakers, and the burner), while the protocol's own keeper, burner and buyer swaps are exact-in and pay the full rate.

RECOMMENDATION

On the exact-out branches, gross the fee up with `fee = amt * FEE_BPS / (BPS - FEE_BPS)` so every swap shape pays 5% of the gross ETH leg.

M-03 MEDIUM FIXED

Leftover USDG ignores `buyCap` and can be swapped all at once

AFFECTED CODE

- `src/robinhood/stock/StockBuyer.sol` , buy(), _runBuys()

DESCRIPTION

`buy` spends at most `buyCap` ETH per round. Stocks quoted in USDG are bought in two steps:

1. Swap their share of the capped ETH into USDG.
2. Swap USDG into each stock.

Step 2 does not spend the USDG that step 1 bought. It spends the contract's whole USDG balance:

```
perUsdg = IERC20(usdg).balanceOf(address(this)) / usdgAssets;
```

If step 2 fails and step 1 succeeds, the USDG stays in the contract, and each later round adds more. The first round where step 2 succeeds then swaps all of it at once. For example, with `buyCap` at 10 ETH and one USDG-quoted stock, five failed rounds leave about 50 ETH worth of USDG, and the next good round swaps about 60 ETH worth in a single trade.

Step 2 can fail while step 1 succeeds when:

- the caller calls `buy` while holding the stock pool's lock, from inside a PoolManager `unlock` (v4) or a pool `flash` (v3), so the stock swap reverts.
- the caller moves the stock pool's price in the same transaction, so the swap misses its oracle floor.
- the stock's feed reports `oraclePaused` , so its price read reverts. This needs no attacker.
- the contract receives USDG from a third party adress.

Unspent ETH does not have this problem: it stays as ETH, and the next round caps it at `buyCap` again.

IMPACT

A swap many times larger than `buyCap` moves the price further. Each swap only has to return the oracle price less `slippageBps` (1% to 15%), so the protocol can lose up to that share of the whole backlog to price impact or a sandwich.

If the backlog grows so large that even the smallest retry cannot fill within `slippageBps` , every attempt fails. Each later round still buys more USDG, so the backlog keeps growing and USDG-quoted stocks stop being bought until the owner changes the retry settings.

RECOMMENDATION

Limit the USDG each stock spends per round to what its ETH share buys at the oracle price. Leftover USDG then drains one share per round, the same way unspent ETH does.

M-04 MEDIUM FIXED

One shared cooldown lets a caller keep chosen stocks out of every `buy`

AFFECTED CODE

- `src/robinhood/stock/StockBuyer.sol`, buy(), _runBuys()

DESCRIPTION

`buy` buys every stock in the basket in one round, and one cooldown covers the whole basket. The round stamps it before any swap runs:

```
if (block.number < uint256(lastBuyBlock) + buyInterval) revert StockBuyer__TooEarly();
// ...
lastBuyBlock = uint64(block.number);
```

Each stock is bought in its own `try`. A stock whose swap fails is skipped, the round still succeeds, and nobody can buy that stock again until the next interval.

The caller can make a chosen stock's swap fail on purpose:

- Push that stock pool's price past the oracle floor, call `buy`, and push the price back, all in one transaction. The floor comes from a TWAP, which does not follow the spot price, so every retry misses it. This costs the pool fee twice on the push: little on a 0.05% v3 pool, a lot on a 5% v4 pool.
- Call `buy` from inside a zero-amount `flash` on that stock's v3 pool. The pool is locked, so the swap reverts with `LOK`. This costs only gas.

The other stocks fill, so `filledWad` is not zero and the caller collects the bounty for them.

IMPACT

A caller who calls first after each interval keeps the chosen stocks out of the basket for as long as they repeat it. Stakers receive the stocks the caller allows, not the basket the owner set. If the caller lets only the cheapest stock fill, a basket of `m` stocks converts about `1/m` of its budget per interval, and the rest of the ETH waits. The caller earns the bounty on the filled stocks while doing it.

Stamping the cooldown only when something fills, or reverting while the PoolManager is unlocked, does not stop this. Some stocks still fill, and neither the price push nor the v3 lock involves the PoolManager.

RECOMMENDATION

Track the cooldown per stock and stamp it only for stocks that filled. Anyone can then buy a skipped stock in the next transaction, because a price push or a lock only lasts inside the caller's own transaction.

M-05 MEDIUM FIXED

The permissionless `buy` caller can sandwich the protocol's own swaps inside a single transaction and capture most of the oracle slippage band on low-fee venues

AFFECTED CODE

- `src/robinhood/stock/StockBuyer.sol`, buy(), buyUsdg(), buyOne()

DESCRIPTION

`buy` is permissionless and caller-timed, and each leg is bounded only by minOut, a floor between 100 and 1500 bps below the TWAP. The 1-hour TWAP reads the same venue pool but does not react to a spot move inside the same transaction, and nothing checks spot against it. A caller's contract can push spot until the leg's output sits just above the floor, call `buy`, and unwind. The push costs the attacker only the pool fee on their own volume: the keeper's leg moves the price further in the same direction, so the attacker unwinds at a better price than they entered. The attack therefore pays whenever a slice exceeds roughly `pool fee × in-range liquidity` of the venue.

The band is sized for the most expensive venue. The contract's own fork measurement of a 2 ETH AAPL buy puts the 0.05% v3 pool at 139 bps total cost, so at 300 bps about 160 bps of each slice is open to the caller, while the caller's own fees come to about 0.1% of the slice. On the 5% v4 venue (689 bps) the caller would pay 5% on each of their own legs, so that venue is not economical to attack.

The capital cannot come from the venue itself. A flash on the same v3 pool locks it, so the leg reverts with `LOK`, and holding the v4 PoolManager unlocked makes the leg revert with `AlreadyUnlocked`. A flash loan from any other pool or lender works, since it only has to span the transaction.

For USDG-quoted assets the ETH to USDG hop and the USDG to stock leg each have their own band, and the stock leg's floor is built on the USDG the hop actually returned (L326). With the global `slippageBps` and the asset's slippage both at 1500 bps, the combined tolerance is $1 - 0.85^2 = 27.75\%$, above the 15% "owner blast radius". Attacking both legs takes a push in each of the two pools. A real price gap inside the TWAP window, such as a market open, adds further room.

IMPACT

On every buy interval, each attackable leg loses up to the band minus the leg's own cost, and the caller keeps that minus their pool fees: about 1.6% of a 2 ETH v3 AAPL slice at 300 bps, and about 13.6% at the 1500 bps ceiling. USDG assets lose up to 27.75% minus the cost of both legs. The caller also collects the full bounty because every leg fills. The loss falls on StockPayouts' stakers. StockBuyer is currently unfunded per `DEPLOYMENTS.md`, which lowers the likelihood but does not remove the issue.

RECOMMENDATION

Before each leg, revert or skip when the venue's spot (`slot0` / `getSlot0`) deviates from the TWAP by more than the venue fee plus about 50 bps. Keep per-asset slippage close to the measured venue cost, give the ETH to USDG hop its own tight slippage, and bound the combined hop and stock tolerance of USDG assets by one band.

M-06 MEDIUM FIXED

Partial fills still pay tax on the full requested amount

AFFECTED CODE

- `src/robinhood/OLYTaxHook.sol`, _beforeSwap(), _afterSwap()

DESCRIPTION

On an exact-input ETH buy, `OLYTaxHook._beforeSwap` takes 5% of the requested input before the pool executes. `_afterSwap` returns without adjusting that fee. If the integration accepts a partial fill, the tax remains based on the full request rather than the completed trade.

IMPACT

A buy requesting 100 ETH can still pay 5 ETH in tax even when the pool trades only a small part of that amount. The trader is charged for unfilled input, and the excess goes to the tax router. This requires a successful partial fill. A reverted swap also reverses the tax.

RECOMMENDATION

Keep the 5% tax proportional to the completed trade and refund any fee charged on unfilled input. If the hook cannot settle that refund, reject partial fills on this path.

M-07 MEDIUM FIXED

Off-chain oracle pause blocks TWAP quotes for stock tokens

OFF-CHAIN ORACLE PAUSE BLOCKS TWAP QUOTES FOR STOCK TOKENS

AFFECTED CODE

- `src/robinhood/oracles/V3TwapPriceProvider.sol`, sqrtPriceX96(), rateX18(), setRoute(), _requireFeedLive()
- `src/robinhood/farm/RHFarmKeeper.sol`, depositToFarm(), removeFarm()
- `src/robinhood/farm/libs/RHFarmKeeperLib.sol`, referenceSqrtPriceX96()
- `src/robinhood/farm/libs/RHFarmKeeperV3Lib.sol`, decreaseLiquidity()
- `src/robinhood/stock/StockBuyer.sol`, buyOne(), _assetRateX18()

DESCRIPTION

Each route in `V3TwapPriceProvider` can carry a `pauseGuard`, and `setRoute` sets it to the stock token for a Robinhood Stock pair. Before quoting, `sqrtPriceX96` and `rateX18` call `_requireFeedLive`, which reverts with `V3TwapPriceProvider__FeedPaused` when the token's `oraclePaused` returns `true`.

`oraclePaused` only means that Robinhood has frozen the token's off-chain Chainlink feed, which it does during a corporate action. It does not stop transfers, mints or swaps. The provider never reads that feed. It prices from the v3 pool's TWAP, and the pool keeps trading. The check blocks quotes because of a source the provider does not use.

While the flag is up, every price read for that stock reverts. `depositToFarm` reads the price through `referenceSqrtPriceX96`, `removeFarm` through `decreaseLiquidity`, and `StockBuyer.buyOne` through `_assetRateX18`. `buy` catches the last revert and skips the stock. Harvests still work, since `_collectFees` reads no price.

For example:

1. `StockBuyer` lists five assets. During cycle N, Robinhood pauses the feed of one of them, stock X, for a corporate action.
2. A keeper calls `buy` with a 1 ETH spend, so `per` is 0.2 ETH. Four buys go through. `buyOne` for stock X reverts with `V3TwapPriceProvider__FeedPaused`, `buy` emits `BuySkipped`, and 0.2 ETH stays in the buyer.
3. The pause runs past the end of cycle N. Every `buy` until then skips stock X the same way, while stock X trades normally in its pool.
4. Robinhood clears the flag in cycle N+1. The next `buy` spends the leftover ETH, and `sync` adds what it buys to the pot of cycle N+1. Stakers credited for cycle N never receive it.

IMPACT

While a stock's feed is paused, `StockBuyer` buys none of that stock and keeps its slice of each budget. A later `buy` spends it, and `sync` credits the result to the cycle accruing at that time. When the pause runs past the end of a cycle, revenue earned in that cycle goes to the stakers of a later one. Stakers credited only for the earlier cycle lose their share, and later stakers receive it on top of their own.

The stock's farm also adds no liquidity, and the owner cannot remove it. Robinhood sets the flag at each corporate action, so this recurs for every guarded stock, and the protocol cannot clear it. The only way around it is for the owner to re-register the route without a guard, by hand, for each pause.

RECOMMENDATION

Remove the `pauseGuard` field and `_requireFeedLive` from `V3TwapPriceProvider`. A paused token already reverts its own transfers with `IsPaused()`, so any swap or liquidity change that moves it fails without a check in the provider.

M-08 MEDIUM ACKNOWLEDGED

Stakers who end a stake just after a cycle's snapshot still collect that whole cycle's payout

AFFECTED CODE

- `src/StockRelay.sol`, snapshotBlockOf(), _collect(), relay()
- `src/robinhood/stock/StockPayouts.sol`, claim()
- `src/StakingVault.sol`, stakeEnd()

DESCRIPTION

A staker who ends their stake just after a cycle's snapshot block is still paid that whole cycle, although they hold nothing for the rest of it.

`_collect` reads each staker's weight once, at `snapshotBlockOf(cycleId)`, the cycle's first block. The cycle's revenue arrives over the next 28 days, and `claim` pays it against that weight. Nothing checks that the staker still holds the stake later in the cycle. Snapshot blocks are fixed at deploy and stake lengths are set in whole days, so a staker can pick a length that matures just after a snapshot. `stakeEnd` then returns the full principal, since the stake is inside the 7-day grace period.

The NatSpec covers the entry side ("you earn from the cycle after you stake") but not the exit side. This breaks the ledger's stated rule that it pays stakers pro-rata.

For example, with the production cycle of 200,880 blocks (about 28 days):

1. One block before cycle 0's snapshot, Alice stakes 100,000 OLY for 113 days and Bob stakes 100,000 OLY for 365 days. Four cycles last 112.006 days, so Alice's stake matures 23 hours after cycle 4's snapshot.
2. At maturity Alice calls `stakeEnd`. She gets all 100,000 OLY back and holds no shares.
3. Cycle 4 earns 10 ETH, all of it after Alice left.
4. Cycle 4 ends and is relayed. `_collect` reads Alice's votes at cycle 4's first block, so `credit` records her full weight.
5. Alice claims 3.90 ETH. Bob, the only staker for the other 27 days, claims 6.10 ETH.

With a 112-day stake, Alice's stake matures 8 minutes before the snapshot. She then gets nothing for cycle 4 and Bob gets all 10 ETH.

IMPACT

Part of each cycle's revenue goes to stakers who left on its first day, and comes out of the stakers who stay. Any staker can do this at no cost beyond choosing a stake length. The gain is one cycle's share per stake and repeats with every new stake. Stakers who do not time their stakes lose up to a cycle on entry and gain up to one on exit, which averages out. A staker who stakes just before one snapshot and leaves just after another keeps the extra cycle.

RECOMMENDATION

In `_collect`, weight each staker by the smaller of their votes at `snapshotBlockOf(cycleId)` and at `cycleEndBlockOf(cycleId)`, so a staker who leaves during a cycle earns nothing for it. `relay` already waits until the end block is in the past. The denominator still comes from `getPastTotalSupply` at the first block, so the leaver's share would stay in `unpaid`. Roll that share into a later cycle or add a sweep.

M-09 MEDIUM FIXED

Free swaps before the OLY pool holds liquidity fill the oracle ring and remove the burner's slippage floor

AFFECTED CODE

- `src/robinhood/OLYTaxHook.sol`, _recordObservation(), _beforeSwap()
- `src/robinhood/oracles/V4TwapPriceProvider.sol`, rateX18(), _twap()
- `src/robinhood/RHBuyNBurn.sol`, buyNBurn()
- `src/robinhood/RHOracleSwapActions.sol`, _swapExactIn()

DESCRIPTION

The OLY pool holds no liquidity between its initialization and the first deposit. In that window a swap moves the price to its `sqrtPriceLimitX96` and trades zero tokens, so a 1 wei swap moves the price anywhere for free and pays no tax. Each such swap still runs `_beforeSwap`, which starts the ring on the first call and writes an observation every 12 seconds after that.

Anyone can call `increaseCardinalityNext`, and each write can move the recorded tick by up to `MAX_ABS_TICK_MOVE`, 9,116 ticks. An attacker can therefore fill the ring with any price for longer than the 30 minute window the burner reads, then swap the spot back so the pool looks untouched when liquidity arrives.

`rateX18` only checks that the oldest observation is older than the window, so it quotes the poisoned average as soon as the pool is seeded. The provider expects readers to revert while a pool is new, but here they do not. `buyNBurn` has no interval wait on the first call and floors the swap at 92% of that rate.

The poison does not fade with time. With no new writes, `observe` extends the last observation with the current tick clamped to within 9,116 ticks of the poisoned one, so the average stays poisoned until enough swaps land. With one swap every 12 seconds, the rate returns to within 8% of spot after 37 minutes. With one swap per minute, it takes 72 minutes, longer than the 1 hour burn interval.

For example:

1. The pool is initialized at tick 0 with no liquidity.
2. The attacker swaps 1 wei with a price limit at tick -400,000, calls `increaseCardinalityNext(key, 255)`, and repeats a 1 wei swap every 12 seconds for 40 minutes. The ring holds tick -400,000 across the whole window. The attacker spends nothing.
3. The attacker swaps 1 wei back to tick 0. Full-range liquidity with 5 ETH on the ETH side is added at tick 0.
4. `rateX18(ETH, OLY)` returns 4 instead of about `1e18`. The floor for a 0.99 ETH burn is 3 wei of OLY.
5. The burner holds 1 ETH. The attacker buys OLY with 2.25 ETH, calls `buyNBurn`, and sells the OLY back. The burner receives 51% of the OLY a fair price would give. The attacker nets 0.217 ETH after the 5% tax on both legs, including the caller incentive.

Against a pool with 10 ETH on the ETH side, the attacker nets 0.048 ETH and the burner receives 77%. Above about 16 ETH, the tax on both legs makes the sandwich lose money. With an honest average, the same front-run makes the burn revert.

IMPACT

Anyone can remove the burner's slippage floor for the first burns after launch, at no cost. Every burn in that period can be sandwiched, and with a launch pool under about 16 ETH on the ETH side the attacker profits, taking up to half of each burn. Each burn spends at most `swapCap`, 1 ETH, which the owner can raise. With no trades the floor stays gone, so a quiet launch exposes the second burn as well.

If liquidity is added without checking the spot price, the same free swaps let the attacker choose the price the first deposit lands at.

RECOMMENDATION

1. At the top of `_recordObservation`, return when `poolManager.getLiquidity(id)` is zero. Swaps in an empty pool then neither start nor write the ring, and `increaseCardinalityNext` reverts until the first swap with liquidity starts it. The ring begins at a price that liquidity backs, and readers revert until it holds a full window of history, as the provider expects. The check adds one read, about 1,200 gas per swap.

The hook cannot know the intended price of the first deposit. Check the spot price before adding the first liquidity, or add it in the same transaction that initializes the pool. As a second guard, have `V4TwapPriceProvider` or `RHBuyNBurn` reject a TWAP that is far from the spot price.

M-10 MEDIUM FIXED

A lagging TWAP lets a caller sandwich a protocol swap at the pre-move price after a real move

AFFECTED CODE

- `src/robinhood/RHOracleSwapActions.sol`, _swapExactIn() (the shared TWAP floor)
- `src/robinhood/farm/RHFarmKeeper.sol`, depositToFarm(), _farmSwap()
- `src/robinhood/stock/StockBuyer.sol`, buy()
- `src/robinhood/RHBuyNBurn.sol`, buyNBurn()
- `src/robinhood/oracles/V3TwapPriceProvider.sol`, rateX18()

DESCRIPTION

Every protocol swap is floored only by the pair's 1-hour TWAP at the minOut line, where `rateX18` comes from `V3TwapPriceProvider.rateX18`. That TWAP does not move within a transaction and lags a real market move for up to a full window. After the token the protocol is buying has fallen against ETH, a caller pushes the pool's spot back to the stale pre-move price, triggers the swap so it fills there instead of at the live price, and sells back. The overpayment is the caller's profit.

The reference itself is stale, so the loss is the market gap on top of the slippage band, and no owner setting bounds it. A spot-against-1-hour-TWAP check does not help, because the caller pushes spot to that same stale value.

The floor is shared, so this reaches every permissionless swap: the farm rebalance in `depositToFarm`, the stock buys in `StockBuyer.buy`, and the OLY buy in `RHBuyNBurn.buyNBurn`. The OLY pool charges a 5% tax on each leg, which makes its own case costly to attack but does not remove it.

Example, a farm rebalance on the NVDA/WETH pool:

1. NVDA falls 10% against ETH. Spot reflects it, but the 1-hour TWAP still reads the pre-move price `T`.
2. A caller buys NVDA, pushing spot back up to `T`, then calls `depositToFarm`.
3. The rebalance swaps the farm's WETH into NVDA at `T`, which the floor accepts because it also reads `T`.
4. The caller sells the NVDA back at the live price and keeps the difference.

An honest swap at the same moment loses almost nothing, and once the TWAP catches up the sandwich no longer pays.

IMPACT

While a real move still lags the TWAP, any caller can make a protocol swap fill at the pre-move price rather than the live one. This opens an extraction path on every permissionless swap, and the amount grows with the size of the swap and the gap. The loss falls on protocol revenue, which funds L1 stakers through StockPayouts.

RECOMMENDATION

Floor every swap against a reference the caller cannot move within one transaction but that tracks the market within a block, such as a short TWAP of about 60 seconds, and reject the swap when live spot deviates from it by more than the pool fee plus a small margin. A check against the same 1-hour TWAP is not enough.

M-11 MEDIUM FIXED

Changing the buy interval or the incentive band re-prices the time already elapsed, so shortening the interval jumps the next bounty to `maxIncentiveBps`

AFFECTED CODE

- `src/robinhood/stock/StockBuyer.sol`, currentIncentiveBps(), buy(), setBuyConfig(), _setBuyConfig(), setIncentiveParams()

DESCRIPTION

`buy` pays its caller a bounty of `budget × currentIncentiveBps()` (L245). `currentIncentiveBps` measures the blocks since `lastBuyBlock` against the current `buyInterval`, and ramps from `baseIncentiveBps` to `maxIncentiveBps` over `2 × buyInterval` blocks once the interval has passed (L390-L398). Both inputs are read live at the time of the buy.

`_setBuyConfig` (L487) and `setIncentiveParams` (L494-L495) overwrite these values without settling the period in progress. The new interval and band are then applied to blocks that elapsed under the old ones. A shorter interval both opens the buy gate early (L238) and moves the elapsed blocks deep into a ramp that is now shorter too. A higher `maxIncentiveBps` is applied to the whole ramp already accrued.

This happens when a buy period is in progress and the owner shortens `buyInterval` or raises the incentive band while ETH sits in StockBuyer. The setter transaction is public, so the first caller after it lands collects the re-priced bounty.

IMPACT

The bounty is paid out of the stakers' buy budget, so every such change hands the next caller ETH that should have bought stock. For example, with a 300-block interval, a 5 ETH cap and a 50 to 500 bps band, the owner shortens the interval to 50 blocks 250 blocks after the last buy. Under the old setting nobody could buy yet, and the first buy at block 300 would pay the 50 bps base, 0.025 ETH. After the change `over` is 200 blocks against a 100-block ramp, so the next caller is paid the full 500 bps, 0.25 ETH. Each change can move up to `(maxIncentiveBps − current bps) × buyCap`, at most 10% of `buyCap`, from stakers to the caller.

- Such action can be backrunned to capture the increase immediately.

RECOMMENDATION

Settle the period before the parameters change: in `setBuyConfig` and `setIncentiveParams`, restart the ramp by setting `lastBuyBlock = block.number` when a period is in progress, or store the incentive accrued so far and ramp only the blocks after the change with the new values.

LOW SEVERITY · 26 FINDINGS

L-01 LOW FIXED

Dead shares inflate `totalWeight` and strand part of every cycle's pot

AFFECTED CODE

- `src/StockRelay.sol` , _collect()
- `src/StakingVault.sol` , _initializeDeadShares()
- `src/robinhood/stock/StockPayouts.sol` , credit(), claim()

DESCRIPTION

`StockRelay._collect` takes the cycle's `totalWeight` from the vault's total voting supply at the snapshot block:

```
totalWeight = vault.getPastTotalSupply(snapshotBlock);
if (totalWeight == 0) revert StockRelay__ZeroTotalWeight();
```

That supply includes the 1,000 `DEAD_SHARES` that `_initializeDeadShares` assigns to `0xdead` at initialization:

```
userCurrentShares[deadAddress] = DEAD_SHARES;
_transferVotingUnits(address(0), deadAddress, DEAD_SHARES);
```

`0xdead` never delegates, so `delegates(0xdead)` stays `address(0)` and these units count toward the total supply but not toward any account's votes. No relay can credit them, yet `StockPayouts.claim` divides every payout by a `totalWeight` that includes them. In each cycle, a `DEAD_SHARES / totalWeight` fraction of every pot can never be claimed.

Subtracting `DEAD_SHARES` alone does not close the gap. During the mint phase, anyone can call `LaunchManager.mint` with `to` set to `0xdead` . `_startStake` then self-delegates `0xdead` , which moves all of its voting units, the dead shares included, into its votes. Once a relay includes `0xdead` , the credited weights of a cycle can exceed the reduced `totalWeight` by up to 1,000, and the cap in `StockPayouts.credit` skips whichever real staker is relayed last.

IMPACT

Each cycle, a small part of every token's pot stays in `StockPayouts` with no way to claim it. The loss per cycle is `pot * DEAD_SHARES / totalWeight` .

RECOMMENDATION

Subtract `DEAD_SHARES` from the total supply when computing `totalWeight`. `DEAD_SHARES` is private in `StakingVault`, so mirror it as a constant in `StockRelay`. Also reject `0xdead` in `users`, so a relay can never credit its votes against the reduced total.

```
totalWeight = vault.getPastTotalSupply(snapshotBlock) - DEAD_SHARES;
if (totalWeight == 0) revert StockRelay__ZeroTotalWeight();
// ...
for (uint256 i; i < n; ++i) {
    address user = users[i];
    if (user == DEAD_ADDRESS) revert StockRelay__DeadAddress();
```

L-02 LOW FIXED

Revenue accrued to a cycle with no stakers can never be paid out

AFFECTED CODE

- `src/StockRelay.sol` , _collect()
- `src/robinhood/stock/StockPayouts.sol` , sync(), credit(), claim()

DESCRIPTION

`sync` adds new revenue to the pot of the current cycle. That pot becomes claimable only after `StockRelay` relays the cycle's weights and `credit` records a non-zero `totalWeight` for it. `_collect` reads the weights at the cycle's snapshot block, which is the cycle's first block, and refuses to relay when no user in the batch holds votes:

```
totalWeight = vault.getPastTotalSupply(snapshotBlock);
if (totalWeight == 0) revert StockRelay__ZeroTotalWeight();
// ...
if (k == 0) revert StockRelay__NoEligibleWeight();
```

If no account is staked at a cycle's snapshot, for example because every staker ended their stake before the cycle began, nothing can ever be relayed for that cycle. The `ZeroTotalWeight` check does not fire, because the 1000 dead shares minted to `0xdead` keep the total supply above zero. The relay reverts with `NoEligibleWeight` instead. A zero-weight message would also fail on the L2, as `credit` rejects a zero `_totalWeight`.

The revenue synced during that cycle can then never leave the contract. `claim` returns early while `totalWeight` is zero, and `sync` never reassigns the amount because it is still counted in `unpaid` :

```
uint256 acc = unpaid[t];
if (bal <= acc) continue;
```

`StockPayouts` has no owner and no function that moves a pot to another cycle.

IMPACT

All revenue that reaches `StockPayouts` during a cycle with no stakers at its snapshot is locked permanently. This requires the staking base to be empty at the first block of a cycle, which is unlikely once the protocol has users, but LP fees and stock purchases keep arriving whether or not anyone is staked.

RECOMMENDATION

Let `StockRelay` report a cycle with no stakers, and let anyone release that cycle's pot to the accruing cycle. For example, relay an empty-cycle message when `getPastTotalSupply` at the snapshot equals the dead-share floor, record it on the L2, and add a permissionless function that clears the pot:

```
function releaseEmpty(uint32 cycleId, address[] calldata tokens) external {
    if (!emptyCycle[cycleId]) revert StockPayouts__CycleNotEmpty();
    for (uint256 i; i < tokens.length; ++i) {
        unpaid[tokens[i]] -= pot[cycleId][tokens[i]];
        pot[cycleId][tokens[i]] = 0;
    }
}
```

The next `sync` then assigns the released amount to the accruing cycle. No one can claim from an empty cycle, so clearing its pot takes nothing from any staker. Alternatively, document that revenue accrued while no one is staked is lost.

L-03 LOW FIXED

The TWAP providers return stale prices after quiet periods or sequencer halts, which breaks the `IPriceProvider` "MUST revert" rule

AFFECTED CODE

- `src/robinhood/oracles/V4TwapPriceProvider.sol`, _twap()
- `src/robinhood/oracles/V3TwapPriceProvider.sol`, _twap()

DESCRIPTION

IPriceProvider requires providers to revert rather than return a stale price, because a bad answer turns off the minimum-output guard. Neither provider checks the age of the newest observation or the sequencer's uptime. On a quiet pool, or right after a Robinhood Chain halt, the TWAP is just the last pre-gap price carried forward.

IMPACT

Floors for burns, farm swaps and stock buys are built on an outdated price. A caller who times the first call after a gap can trade against the difference, on top of the configured slippage band.

RECOMMENDATION

Revert when the newest observation is older than a maximum age, or add a spot-vs-TWAP deviation check.

L-04 LOW FIXED

The default farm swap slippage (5%) is below the taxed OLY/ETH pool's own 6% cost, so an OLY farm can never rebalance

AFFECTED CODE

- `src/robinhood/farm/libs/RHFarmKeeperLib.sol`, addFarm()
- `src/robinhood/farm/RHFarmKeeper.sol`, _depositToV4Farm()

DESCRIPTION

A farm enabled with `swapSlippage = 0` gets the 500 bps default. A v4 farm rebalances through its own pool, which on the OLY/ETH pool costs the 1% LP fee plus the hook's fixed 5% tax, about 595 bps. The output can therefore never reach `rate × 0.95`, every ladder rung fails even when spot equals the TWAP, and shrinking the size does not help because the tax is proportional. The failures are silent. The deploy script checks slippage against venue cost for the burner, but not for farms.

IMPACT

The OLY farm never converts its single-sided ETH, so its liquidity stays near zero and protocol ETH sits idle, earning nothing, until the owner notices and retunes it.

RECOMMENDATION

Require `swapSlippage` to exceed the venue cost (LP fee plus hook tax) when a farm is enabled or updated, and stop mapping zero to a default that is below that cost.

L-05 LOW FIXED

TWAP routes are wired to pools whose observation rings are smaller than the 1-hour window, so an griefer can make price reads revert

AFFECTED CODE

- `src/robinhood/oracles/V3TwapPriceProvider.sol`, _setRoute()
- `src/libs/V3OracleLib.sol`, getTwapSqrtPriceX96()
- `script/robinhood/RHPools.sol`, pool list

DESCRIPTION

Every price read goes through `getTwapSqrtPriceX96`, which reverts with `V3OracleLib__TooShortPoolCardinality` when the pool's oldest stored observation is younger than `twapPeriod` (3600 s). `_setRoute` only checks that the ring reaches back 1 hour at registration time. It never checks that the ring has enough slots to keep reaching back 1 hour. RHPools.sol itself records that 11 of the 13 stock pools have fewer than 3600 slots (META 1400 up to SPCX 3100). The window works today only because these pools are quiet and each slot covers a long gap.

A v3 pool stores at most one observation per second. An in-range mint or burn with non-zero liquidity always writes one (a swap writes only if it changes the tick). An attacker who mints and burns a 1-wei in-range position every second makes each slot cover 1 s. After `cardinality` seconds the whole ring is overwritten. For META, the oldest observation is then about 1400 s old, so every read reverts.

IMPACT

While the attack runs, `rateX18` and `sqrtPriceX96` revert on the affected routes. The reads keep reverting for up to `3600 - cardinality` seconds after it stops (about 2200 s for META). This blocks `depositToFarm` and the owner's `removeFarm` exit (both read `sqrtPriceX96` for slippage). It also makes `StockBuyer` silently skip the affected stocks, because each buy runs in a `try/catch`. The cost is L2 gas plus dust, which the burn returns. No funds are lost, and the attacker does not profit.

Raising the cardinality is a slow fix. New slots fill only as the index wraps onto them, at one per second while the attack runs, so going from 1400 to 3600 takes about an hour and about 20k gas per slot.

RECOMMENDATION

In `_setRoute`, require `observationCardinality >= twapPeriod` (the filled cardinality, not just `observationCardinalityNext`). Have the deploy script call `increaseObservationCardinalityNext(twapPeriod)` on each pool well before registering its route.

L-06 LOW FIXED

`claimable` and `previewPayout` overstate payouts during a shortfall

AFFECTED CODE

- `src/robinhood/stock/StockPayouts.sol`, claimable(), previewPayout(), claim()

DESCRIPTION

`claimable` returns a staker's nominal share of a cycle's pot, and `previewPayout` does the same for a given weight:

```
/// @notice Amount `staker` can currently claim from `cycleId` for `token`. UI helper.
function claimable(uint32 cycleId, address staker, address token) external view returns
(uint256) {
    if (claimed[cycleId][staker][token]) return 0;
    uint256 W = totalWeight[cycleId];
    if (W == 0) return 0;
    return (pot[cycleId][token] * weightOf[cycleId][staker]) / W;
}
```

When the token balance falls below `unpaid[token]`, `claim` pays `amt * held / owed` instead of `amt`. Neither view applies this haircut. For a staker owed `50e18` with the balance at half of `unpaid`, `claimable` returns `50e18` while `claim` pays `25e18`. With the balance at zero, `claimable` still returns `50e18` while `claim` pays nothing, which `test_dustHaircutLeavesTheEntitlementOpen` asserts. The NatSpec of `claimable` promises the amount a staker "can currently claim", which is the haircut figure.

IMPACT

No contract in `src/` calls either view, so on-chain accounting is unaffected. Front ends and integrators that read these views show stakers an amount larger than `claim` will pay whenever a token is under-backed.

RECOMMENDATION

Apply the same haircut in both views so they return what `claim` would pay at that moment:

```
uint256 amt = (pot[cycleId][token] * weightOf[cycleId][staker]) / W;
uint256 owed = unpaid[token];
uint256 held = token == NATIVE ? address(this).balance :
IERC20(token).balanceOf(address(this));
return held < owed ? Math.mulDiv(amt, held, owed) : amt;
```

If the nominal figure is intended, document both views as nominal entitlements and add a separate view for the amount after the haircut. Update `test_dustHaircutLeavesTheEntitlementOpen` to match either choice.

L-07 LOW ACKNOWLEDGED

The `StockBuyer` owner can drain staker ETH through a fake price provider, although the NatSpec promises there is no owner extraction path

AFFECTED CODE

- `src/robinhood/stock/StockBuyer.sol`, setPriceProvider(), addAsset(), setEthUsdgVenue(), setBuyConfig()
- `src/robinhood/RHOracleSwapLib.sol`, executeSwap(), settleV4Swap()

DESCRIPTION

The NatSpec says "There is no fund-extraction path for the owner", and that the 15% slippage cap limits how much the owner can take ("blast radius"). But that cap only works if the price is honest, and the owner picks both the price source and the pool:

1. The owner sets a fake price provider that says 1 ETH buys almost no stock. `setPriceProvider` does not check the price it returns, so the minimum output of every buy becomes about 0.
2. The owner creates their own v4 pool with a terrible price and makes it the asset's venue (`removeAsset` + `addAsset`). Any v4 pool is accepted.
3. The owner calls `buy()`. All the ETH is swapped into their pool for a few wei of stock, and the owner removes the liquidity and keeps the ETH. `setBuyConfig` lets the owner remove the per-buy cap first, so one call is enough.

IMPACT

The owner, or anyone who steals the owner's key, can take all the ETH in `StockBuyer`, and every later deposit the same way. This ETH is tax revenue meant for stakers. Funds already in `StockPayouts` are safe.

RECOMMENDATION

Make the provider immutable or timelocked, bind each venue to the pool the provider quotes, and check a new provider's rates against the old one within a bound. Or either remove the documented guarantee.

L-08 LOW FIXED

Native-in v4 swaps fail when another currency is left synced in the PoolManager

AFFECTED CODE

- `src/robinhood/RHOracleSwapLib.sol`, settleV4Swap()
- `src/robinhood/RHOracleSwapActions.sol`, _swapExactIn()

DESCRIPTION

`settleV4Swap` pays a native-in v4 swap by calling `settle` with `msg.value` and no prior `sync`. ERC-20 input goes through `sync`, a transfer, and `settle`.

```
if (owed != 0) {
    if (tokenIn == address(0)) {
        pm.settle{value: owed}();
    } else {
        pm.sync(Currency.wrap(tokenIn));
        IERC20(tokenIn).safeTransfer(address(pm), owed);
        pm.settle();
    }
}
```

`PoolManager.settle` takes no currency argument. It reads the currency last passed to `sync` from transient storage. If that currency is native, it credits `msg.value`. Otherwise it reverts with `NonzeroNativeValue` whenever `msg.value` is non-zero. The slot lasts for the whole transaction, and `sync` has no lock check, so anything that runs earlier in the same transaction can leave an ERC-20 synced: the venue hook during `beforeSwap` or `afterSwap`, or a caller that calls `PoolManager.sync` directly before entering the protocol. In either case the native `settle` reverts.

The retry ladder in `_swapExactIn` catches every attempt, so the failure surfaces as a zero output rather than a revert.

IMPACT

A venue hook that syncs a currency without settling it makes every native-in swap on that venue fail for every caller until the owner changes the venue. Any caller can force the same failure inside their own transaction for the cost of gas. No funds leave protocol control.

RECOMMENDATION

Call `sync` on the input currency before settling on both branches, as `DeltaResolver._settle` in v4-periphery does. `sync` on `address(0)` clears the stored currency, so the native `settle` then credits `msg.value`.

```
if (owed != 0) {
    pm.sync(Currency.wrap(tokenIn));
    if (tokenIn == address(0)) {
        pm.settle{value: owed}();
    } else {
        IERC20(tokenIn).safeTransfer(address(pm), owed);
        pm.settle();
    }
}
```

L-09 LOW FIXED

Anyone can trigger the keeper's protocol-owned swaps and sandwich them in the same transaction, because the only protection is a TWAP-relative floor with no spot check

AFFECTED CODE

- `src/robinhood/farm/RHFarmKeeper.sol`, depositToFarm(), swapNativeForFarm(), _swapV3EthForFarmToken(), _swapV4EthForFarmToken()
- `src/robinhood/RHOracleSwapLib.sol`, executeSwap()

DESCRIPTION

`depositToFarm` and `swapNativeForFarm` are permissionless, so any caller picks the moment the keeper swaps pooled protocol funds. The only bound on these swaps is the floor `amountOut >= attempt × rateX18 × (10_000 − slippage) / 10_000`, where `rateX18` is the TWAP. Nothing compares the traded pool's live spot price with that reference, and the TWAP does not move inside one transaction. A caller can therefore push spot down to just above the floor, trigger the swap, and swap back, all atomically.

The intermediary ETH-to-token swap of a non-native farm is the cleanest case: it runs in a different pool from the farm's LP, and nothing afterwards re-checks that pool's price, so the whole `intermediarySwapSlippage` band is exposed. The rebalance swap in the farm's own pool is bounded more tightly, because the LP add that follows in `depositToFarm` derives `amount0Min` / `amount1Min` from the TWAP and `liquiditySlippage`. The attacker must restore spot before that add or the whole call reverts, so the exposure there is about min(`swapSlippage`, LP band). The rebalance swap through the taxed OLY/ETH pool is not economical to attack, because the attacker would pay the hook's 5% tax on both legs of their own, larger, manipulation volume.

IMPACT

On every swap the keeper receives up to the slippage band less than the TWAP value, and the caller keeps that shortfall minus the LP fees on their own push-and-return volume. The attacker's price impact is not a cost, because the keeper's swap pushes the price further in the same direction and the attacker unwinds at the better price. The attack therefore pays whenever the keeper's swap exceeds roughly `pool fee × in-range liquidity` of the pool being moved, which is most likely for the thin intermediary pools of non-native farms. Keeper calls pay their caller nothing, so nobody is paid to deposit small amounts early, and an attacker can let a farm's balance grow until that condition holds. It repeats on every ETH inflow and harvest reinvestment, and the loss falls on the L1 stakers whose StockPayouts revenue is funded by these farms.

RECOMMENDATION

Before every keeper swap, require the traded pool's live spot (`slot0` / `getSlot0`) to be within a tight deviation of the provider's TWAP (pool fee plus a small margin).

L-10 LOW FIXED

Farm deposits revert once the farm's own swap moves the pool price by more than `liquiditySlippage`

AFFECTED CODE

- `src/robinhood/farm/RHFarmKeeper.sol`, depositToFarm(), _doDepositToFarm(), receive(), setLiquiditySlippage(), setSwapSlippage()
- `src/robinhood/farm/libs/RHFarmKeeperV4Lib.sol`, addLiquidity(), _prepareIncreaseUnlockDate()
- `src/robinhood/farm/libs/RHFarmKeeperV3Lib.sol`, addLiquidity(), _calculateTWAPBounds()
- `src/robinhood/farm/libs/RHFarmKeeperLib.sol`, validateFarmParams(), distributeEth()
- `src/robinhood/RHOracleSwapActions.sol`, _swapExactIn()

DESCRIPTION

Farm revenue arrives on one side only, as ETH split across farms. `depositToFarm` sells about half of it through the farm's own pool, and the swap is accepted as long as its output is within `swapSlippage` of the oracle quote. The add step then prices the position at the oracle, a TWAP that has not moved, and caps each token amount at that price plus `liquiditySlippage`. The mint runs at the moved pool price, needs more of the sold token than the cap, and `PositionManager` reverts with `MaximumAmountExceeded`. V3 farms fail the same way on the oracle-priced minimums.

For example, a V4 farm's pool holds 50 ETH and 50,000 TOKEN in full-range liquidity (1 ETH = 1,000 TOKEN). `liquiditySlippage` is 1% and `swapSlippage` is the default 5%. The farm has 2 ETH of revenue waiting:

1. `depositToFarm` sells about 1 ETH for 979 TOKEN, 2.25% below the oracle quote. The swap passes the 5% check.
2. The pool price drops to 961 TOKEN per ETH. The TWAP still reads 1,000.
3. The add step plans the position at 1,000 and caps its ETH at 0.979.
4. At 961, the same position needs 0.989 ETH, so the mint and the whole deposit revert.

With 1 ETH of revenue, the same deposit succeeds. In general, a deposit reverts once the waiting revenue is worth more than `liquiditySlippage` of the liquidity around the current price, whenever `liquiditySlippage` is below `swapSlippage`. `validateFarmParams` allows that pairing. The pool's total value does not matter. If 80% of the same pool sat in a range away from the current price, deposits would revert above 0.2 ETH. If the farm were the only LP, they would revert above about 1% of the farm's own position.

The farm does not recover on its own. Revenue keeps landing on the same side, and `depositToFarm` always deposits the whole balance, so every later call reverts. Anyone can push a farm past the limit by sending ETH to `receive`.

IMPACT

Revenue stops earning LP fees for stakers until the owner raises `liquiditySlippage` through `setLiquiditySlippage`, which also weakens the add step's protection against sandwiching. Shallow pools, pools whose liquidity sits away from the current price, and pools where the farm is the main LP hit the limit first. No funds leave protocol control.

RECOMMENDATION

Stop the add step from treating the farm's own swap as slippage. Either cap the swap so that it cannot move the price by more than `liquiditySlippage`, or check the add against the post-swap pool price after validating that price against the oracle. At a minimum, require `liquiditySlippage >= swapSlippage` in `validateFarmParams` and in both setters. Also add a maximum amount to `depositToFarm`, so that a large balance can be deposited over several calls.

L-11 LOW FIXED

A staker can make harvest syncs fail from inside `claim` and push that revenue into the next cycle

AFFECTED CODE

- `src/robinhood/stock/StockPayouts.sol`, claim(), sync()
- `src/robinhood/farm/libs/RHFarmKeeperLib.sol`, distributeRewards(), syncPayouts()
- `src/robinhood/farm/RHFarmKeeper.sol`, harvestFarm(), harvestAllFarms()
- `src/robinhood/stock/StockBuyer.sol`, buy(), _runBuys()

DESCRIPTION

A staker can trigger a harvest from inside their own `claim`. The harvest's `sync` then fails, and the staker chooses which cycle receives the revenue.

`claim` holds the `StockPayouts` lock while it transfers each token. `sync` accepts any token, so a staker can sync a token they deployed into a cycle where they hold weight, and claim it later. That token's `transfer` calls back into the attacker. A contract set as the staker's destination gets the same callback from an ETH claim.

From the callback, the attacker calls `harvestFarm`, `harvestAllFarms` or `StockBuyer.buy`. All three are permissionless. Each sends funds to `StockPayouts` and then calls `sync`, which reverts on the lock. `syncPayouts` and `_runBuys` catch the revert and only emit an event. The harvest succeeds, but no cycle owns the funds. The next `sync` books them to whichever cycle is accruing at that time.

Any staker could undo this by calling `sync` before the cycle ends, so the attacker makes the sync fail in the last block of cycle `N`. On this chain `block.number` is the L1 block number, so others have about 12 seconds to react. The attacker calls `sync` in the first block of `N + 1`. That block is also the `N + 1` weight snapshot, so stakers who left during `N` get no share.

For example, cycle `N` has weights Eve 100, Alice 400 and Bob 500. Bob unstakes during `N`, so `N + 1` has Eve 100 and Alice 400. A farm comes due with 10 ETH of fees, of which 6.8 ETH goes to stakers:

1. Earlier, in a cycle where she had weight, Eve sent 1e18 of her own token to `StockPayouts` and called `sync`. That cycle was credited, and she has not claimed the token.
2. In the last block of `N`, Eve calls `claim` for that token. `StockPayouts` takes its lock and calls the token's `transfer`, which calls `harvestFarm`.
3. The harvest sends 6.8 ETH to `StockPayouts` and calls `sync`, which reverts with `ReentrancyGuardReentrantCall`. `syncPayouts` emits `FarmKeeper_PayoutsSyncFailed`, and the harvest completes.
4. Cycle `N + 1` begins. Eve calls `sync([ETH])`, and the 6.8 ETH lands in the pot of `N + 1`.
5. Eve gets 1.36 ETH (20%) instead of 0.68 ETH (10%). Bob earned 3.4 ETH during `N` and gets nothing.

`StockBuyer.buy` runs once per `buyInterval`, so a buy is almost always available near the end of a cycle, and the attacker also keeps the caller bounty. A farm harvest moves 28 days of that farm's fees at once, but the farm must come due near the end of a cycle before anyone else harvests it. `harvestAllFarms` then harvests every due farm in one call.

IMPACT

Revenue earned during one cycle goes to the next cycle's stakers. The attacker acts only at boundaries where their share grows, such as after large unstakes, so the shift does not even out across cycles. Each attack moves a full `StockBuyer` budget or a farm's full staker share. Setup costs one token deployment and one `sync`, and the attacker can prepare many tokens at once. No funds leave protocol control.

RECOMMENDATION

Remove `nonReentrant` from `sync`. It moves no funds, and `claim` updates all its state before it transfers, so `sync` can safely run inside `claim`. A harvest that runs during a claim then books its revenue to the current cycle.

L-12 LOW FIXED

An outside balance drop breaks the keeper's shared books

AFFECTED CODE

- `src/robinhood/farm/libs/RHFarms.sol` , Farm.currency0Balance
- `src/robinhood/farm/libs/RHFarmKeeperLib.sol` , distributeRewards(), removeFarmAssets()
- `src/robinhood/farm/RHFarmKeeper.sol` , harvestFarm(), removeFarm()

DESCRIPTION

`RHFarmKeeper` keeps a separate book for each farm: `currency0Balance` , `currency1Balance` and the accumulated fees. All farms that use the same token share one real `balanceOf(keeper)` . Inflows are booked once by measuring the balance change, and the books are never checked against the live balance again. When a farm is harvested or removed, `distributeRewards` transfers exactly the booked amount, with no cap at what the keeper actually holds.

If the keeper's balance of a token falls without the keeper doing anything, the books add up to more than the keeper holds. An issuer wiping or burning a held balance would do this, for example **USDG** `wipeFrozenAddress` / `decreaseSupplyFromAddress` or a stock-token `adminBurn` . Nothing in the contract notices.

IMPACT

The first farm to harvest or be removed is paid in full out of the shared balance. The whole shortfall then falls on the last farm holding that token. That farm's `safeTransfer` reverts, so its harvest and `removeFarm` both revert permanently. Its other token is frozen along with it. The stakers who should receive that farm's fees and principal lose them, and the owner's only ways out are donating the missing amount or upgrading the proxy. This is Low because it needs an issuer action against the keeper's balance.

RECOMMENDATION

Track a total booked amount per token. Before paying out, clamp the booked amount to the keeper's live balance minus what the other farms have booked, or scale all books for that token down pro rata. In `removeFarmAssets` , cap each transfer at the current balance and emit the shortfall. Removal then always completes and the loss is visible on-chain.

L-13 LOW PARTLY FIXED

A haircut claim is booked as fully paid, and later revenue silently fills the hole

AFFECTED CODE

- `src/robinhood/stock/StockPayouts.sol`, claim(), sync()
- `src/robinhood/farm/libs/RHFarmKeeperLib.sol`, syncPayouts()

DESCRIPTION

When `StockPayouts` holds less of a token than it owes (`held < unpaid[t]`), claim pays only `amt * held / owed`. It still marks the claim as fully done: it sets `claimed = true`, adds the full `amt` to `allocated` and subtracts the full `amt` from `unpaid`. Nothing records the part that wasn't paid.

Two related flaws make this worse:

- **Solvency is checked on the raw balance.** `held` is the raw balance, so it includes revenue that has arrived but hasn't been synced yet. That revenue belongs to the cycle currently accruing, but it can hide a real shortfall from the solvency check.
- **New revenue fills the old hole first.** `sync` only books `bal - unpaid`. Any revenue that arrives during a shortfall goes to filling the old deficit instead of the cycle that earned it. This includes the keeper's harvest delivered through `syncPayouts`.

IMPACT

The shortfall isn't shared fairly. The order of claims and syncs decides who pays for it:

- Anyone who claims during the shortfall loses the difference forever. When the balance recovers, the surplus is synced into the current cycle, not returned to them.
- While unsynced revenue is sitting in the contract, old-cycle claimants are paid in full out of the new cycle's money. The new cycle's stakers end up covering the old cycle's loss.
- Keeper harvests delivered during a shortfall never reach the cycle they arrived in.

This needs an external shortfall to start, such as an issuer wipe or burn of the ledger's balance. Once that happens, every staker in the affected cycles and the next cycle is exposed.

RECOMMENDATION

Track a deficit per token, recorded at the moment the shortfall appears. Measure `held` against synced value only, capped at `unpaid` before any new revenue is added. On the haircut branch, keep `amt - pay` owed to the claimant instead of marking the claim done. Then book new arrivals to the accruing cycle, and repay old cycles only from a recorded recovery.

L-14 LOW FIXED

The V3 ETH-swap path picks the WETH side of the intermediary pool without checking that exactly one side is flagged

AFFECTED CODE

- `src/robinhood/farm/libs/RHFarmKeeperV3Lib.sol`, getSwapTokens(), processEthSwapResult()
- `src/robinhood/farm/RHFarmKeeper.sol`, _swapV3EthForFarmToken(), setWethToken()

DESCRIPTION

When an intermediary pool is configured, `validateIntermediaryPoolTokens` requires exactly one of its tokens to be flagged in `isWethToken`. The swap paths never check this again. Both `getSwapTokens` (L373) and `processEthSwapResult` (L349) pick the WETH side with `isWethToken[intToken0] ? intToken0 : intToken1`. The only guard (L374) catches a zero address, not the wrong token. `setWethToken` can change the flags after the pool is stored. If WETH is `intToken0` and it is unflagged, the selection falls through to `intToken1`, the farm token. If WETH is `intToken1` and the farm token is flagged, the selection picks `intToken0`, the farm token. In both cases `_swapV3EthForFarmToken` calls `IWETH9(token).deposit{value: ethAmount}` on the farm token.

IMPACT

For a standard ERC-20, the `deposit` call reverts. Every `depositToFarm` and `swapNativeForFarm` for the affected farm reverts while the farm holds ETH, so its revenue share idles until the owner restores the flags. The revert also comes from the token rather than `FarmKeeper_TokenNotRegistered`, which hides the misconfiguration.

RECOMMENDATION

In both `getSwapTokens` and `processEthSwapResult`, revert unless exactly one side is flagged:

```
if (isWethToken[intToken0] == isWethToken[intToken1]) revert FarmKeeper_TokenNotRegistered();
```

L-15 LOW FIXED

Stock tokens sent directly to `StockBuyer` are permanently stuck

AFFECTED CODE

- `src/robinhood/stock/StockBuyer.sol`, buyOne(), sweepForeign(), removeAsset()
- `src/robinhood/RHOracleSwapLib.sol`, executeSwap()

DESCRIPTION

When anyone transfers a registered stock token straight to `StockBuyer`, the tokens can never leave. No buy forwards them to `StockPayouts`, and the owner cannot sweep them.

`StockBuyer` moves stock tokens in three ways, and none of them touch a balance that was already there:

1. `buyOne` sends only `out` to `StockPayouts` (StockBuyer.sol#L376). `executeSwap` measures `out` as the change in balance during the swap (RHOracleSwapLib.sol#L49), so any earlier balance stays behind.
2. `sweepForeign` reverts on any token in `everRegistered` (StockBuyer.sol#L507). `addAsset` sets that flag and nothing clears it, `removeAsset` included.
3. A stock token is never a swap input. `buyOne` always spends USDG or native ETH (StockBuyer.sol#L370).

In normal use, `buyOne` forwards each purchase in the same transaction, so `StockBuyer` holds no stock between transactions. The `everRegistered` guard therefore only ever blocks tokens that arrived by direct transfer.

IMPACT

Anyone who sends a registered stock token to `StockBuyer` by mistake loses it for good. `StockPayouts` expects revenue as plain transfers, so a sender that picks the wrong contract is plausible. Stakers lose nothing, and no one can profit from the stuck balance.

RECOMMENDATION

When buying send the whole balance of the stock token instead of the amount returned by the swap.

L-16 LOW FIXED

Relay overpayments remain at an inaccessible L2 address

AFFECTED CODE

- `src/StockRelay.sol`, relay()

DESCRIPTION

`StockRelay.relay` promises to refund excess ETH, but forwards all `msg.value` to the Inbox. Under the reviewed Nitro fee accounting, refunds cover the declared submission and gas budgets. Any deposit above `maxSubmissionCost + gasLimit * maxFeePerGa`s remains at StockRelay's aliased address on L2.

IMPACT

A relayer who sends extra ETH loses access to that remainder. StockRelay has no function to recover it, and later relay calls must fund their own declared budgets. Unused gas within the declared budget still follows the normal refund path. The stuck ETH is the relayer's own overpayment, not staker revenue.

Recommendation Require `msg.value` to equal the declared budget, or forward only that budget and return the remainder to the caller on L1. Update the refund documentation to match.

L-17 LOW FIXED

Quiet pools can lose working TWAP quotes

AFFECTED CODE

- `src/oracle/TruncatedOracle.sol`, transform()
- `src/libs/V4OracleLib.sol`, consult(), getTwapSqrtPriceX96()
- `src/robinhood/oracles/V4TwapPriceProvider.sol`, _twap()

DESCRIPTION

`TruncatedOracle.transform` shifts elapsed seconds by 128 bits inside a `uint144`. This discards all but the lowest 16 bits of elapsed time before dividing by liquidity. After a sufficiently long gap between observations, the computed cumulative can decrease across a query window. `V4OracleLib.consult` then reverts on checked subtraction. Price reads perform this calculation even though they discard the liquidity result. The same checked subtraction also reverts if the `int48` tick accumulator wraps, which takes years at normal prices.

IMPACT

A quiet pool can temporarily lose working TWAP quotes despite having enough price history and an unchanged price. Operations that require those quotes can consequently revert until a later observation makes the window usable again. Funds are not stuck.

RECOMMENDATION

Use wider arithmetic for the shift and division, and handle accumulator wrap consistently in the reader. Avoid computing the liquidity statistic when a caller only needs the price.

L-18 LOW FIXED

Skipped oracle updates misweight price changes

AFFECTED CODE

- `src/robinhood/OLYTaxHook.sol`, _recordObservation()
- `src/oracle/TruncatedOracle.sol`, transform()

DESCRIPTION

`OLYTaxHook._recordObservation` skips updates less than 12 seconds apart. Prices can change between updates, but the oracle does not record how long each price held. The next write or extrapolated read applies the current tick to the entire interval since the last observation. This happens even when price changes stay within the intended tick clamp.

IMPACT

The TWAP can give a short-lived price too much weight. Contracts using that quote can calculate swap minimums and liquidity amounts from an incorrectly weighted price. No material loss has been shown from this on the current pools.

Recommendation Account for the time spent at each price even when stored observations remain 12 seconds apart. Keep the intended tick clamp separate from the observation storage schedule.

L-19 LOW FIXED

Rebalancing swap on v4 farms ignores the protocol fee and swaps too little

AFFECTED CODE

- `src/robinhood/farm/libs/RHFarmKeeperLib.sol`, calculateOptimalSwap(), _effectiveFee()
- `src/robinhood/farm/RHFarmKeeper.sol`, depositToFarm()

DESCRIPTION

Once the v4 protocol fee controller sets a protocol fee on a pool, every `depositToFarm` on a farm in that pool swaps too little to balance the two tokens.

`calculateOptimalSwap` sizes the rebalancing swap as `gap / (2 - fee)` (L781-L782), so that both sides hold equal value after the pool takes its fee. It takes `fee` from `_effectiveFee`, whose NatSpec promises "the fee actually charged by the pool" (L785). The function returns `poolKey.fee`, or `slot0.lpFee` for a dynamic-fee pool (L800-L805). Neither value includes the protocol fee.

The v4 PoolManager charges the protocol fee on top of the LP fee, as `protocolFee + lpFee - protocolFee * lpFee / 1e6` (Pool.sol#L286-L307). The controller sets it per direction, up to `1000` pips, or 0.1% (ProtocolFeeLibrary.sol#L8). v3 farms do not have this problem, because a v3 pool takes its protocol fee out of the LP fee and the swapper pays the same total (UniswapV3Pool.sol#L682-L686).

Example on a v4 farm with a 0.3% LP fee, a 0.1% protocol fee, a price of 1, and no price impact:

1. The farm holds 1,000,000 token0 and no token1. `depositToFarm` runs and `_effectiveFee` returns `3000`.
2. `calculateOptimalSwap` sets the swap to 1,000,000 * 1e6 / (2e6 - 3000) = 500,751 token0.
3. The PoolManager charges `3997` pips, not `3000`. The farm receives 498,750 token1 and keeps 499,249 token0.
4. `addLiquidity` deposits 498,750 of each token, and 499 token0 stays in `currency0Balance` (RHFarmKeeperV4Lib.sol#L242-L249).

IMPACT

Each deposit on an affected v4 farm leaves about half the protocol fee, times the gap, out of the position. That is at most about 0.05% of the gap. The surplus waits in the farm balance until the next `depositToFarm`, which anyone can call. The deposit does not revert, and no user can trigger or profit from the shortfall, since only the Uniswap protocol fee controller sets the fee. No funds leave protocol control.

RECOMMENDATION

Include the protocol fee in `_effectiveFee` for v4 farms. Read `protocolFee` and `lpFee` from the same `getSlot0` call, select the protocol fee for the swap direction from `result.swapToken0`, and combine the two with `ProtocolFeeLibrary.calculateSwapFee`. Read `slot0.lpFee` for static-fee pools too, since a static pool stores `poolKey.fee` there at initialization. Update the NatSpec if the fee is meant to stay LP-only.

L-20 LOW FIXED

Buy-and-burn floor ignores the 5% OLY tax, leaving about 2% of its 8% slippage tolerance

AFFECTED CODE

- `src/robinhood/RHBuyNBurn.sol`, buyNBurn(), _setConfig()
- `src/robinhood/RHOracleSwapActions.sol`, _swapExactIn()

DESCRIPTION

`buyNBurn` buys OLY on the OLY/ETH v4 pool, whose hook is `OLYTaxHook`. Its swap floor does not net out the hook's 5% tax, so the tax and the 1% LP fee use up 6 of the 8 points of slippage tolerance before the price moves.

`_swapExactIn` sets the floor to `amountIn * rateX18 * (1 - slippageBps)` (L159). `rateX18` is the 30-minute TWAP mid price from `V4TwapPriceProvider` (L96-L110), which includes neither the tax nor the LP fee. On an exact-in buy, `OLYTaxHook._beforeSwap` takes 5% of the ETH input before the swap runs (L101-L106), and the pool takes its 1% fee from the rest. Both costs scale with the swap size, so a smaller retry attempt fails the same way.

The deploy script checks once that `BNB_SLIPPAGE_BPS` exceeds the tax plus the LP fee, 600 bps (L205-L206). `_setConfig` does not repeat the check and accepts any value from 100 to 1500 bps. With spot equal to the TWAP, every call reverts at 595 bps or less.

IMPACT

`buyNBurn` reverts whenever OLY spot sits more than about 2.2% above its 30-minute TWAP, as it does after any large buy until the TWAP catches up. Any `slippageBps` at or below 595, which `setConfig` accepts, blocks buy-and-burn until the owner raises it. The ETH stays in the contract. No funds leave protocol control.

RECOMMENDATION

Remove the venue's own cost from the expected output before applying `slippageBps`, so that `slippageBps` covers only price movement. For the OLY pool, multiply the TWAP amount by `(10_000 - FEE_BPS) / 10_000` and by `(1e6 - fee) / 1e6`. Alternatively, move the deploy-script check into `_setConfig` and `_setVenue`, and require a margin above the venue's cost.

L-21 LOW ACKNOWLEDGED

The 28-day harvest gate runs on a wall clock unrelated to the payout ledger's L1-block cycles, so each farm's fees are booked up to a full cycle late

AFFECTED CODE

- `src/robinhood/farm/RHFarmKeeper.sol`, harvestFarm(), harvestAllFarms()
- `src/robinhood/farm/libs/RHFarmKeeperLib.sol`, harvestFarm()
- `src/robinhood/stock/StockPayouts.sol`, sync()

DESCRIPTION

LP fees reach `StockPayouts` only when a farm is harvested (or removed). A harvest is allowed once `block.timestamp >= lastHarvestTime + 28 days` (HARVEST_CYCLE). It sends the 68% staker share to the ledger, calls `sync`, and resets `lastHarvestTime` to the call time. `sync` books whatever has arrived into the cycle accruing now (L275). The ledger's cycles are counted in L1 blocks (currentCycle()), and each cycle's weights are read at its first block. Nothing ties the harvest clock to this grid, so the part of a lump earned before the last boundary is booked into the cycle after the one that earned it.

The ledger accepts this kind of lag by design and keeps it small through frequent delivery. `StockBuyer` notes that value unspent at a boundary rolls into the next cycle, and that its incentive ramp bounds this to what accumulates between calls. The keeper's 28-day gate forbids frequent delivery, so its lag can reach a whole cycle. Harvests are permissionless and their due time is public. When a farm comes due shortly before a boundary, any caller can book its lump into the earlier cycle by calling before the boundary. otherwise it falls into the later one.

IMPACT

Each farm's staker share, 68% of its LP fees, is booked up to one full cycle after it was earned, depending on where its harvest falls in the cycle. Stakers who join or leave between two snapshots gain or lose their part of that lagged amount. An L1 stake has a minimum term of 88 days and can end early only after half of it, at a 65% penalty, so for a long-term staker the gain on entry and the loss on exit largely cancel, and no fees are lost overall. Near a boundary, the first caller decides which cohort receives a farm's whole lump. Because every harvest resets the clock to its call time, harvests that routinely run days late also drift until a cycle receives no harvest from that farm.

RECOMMENDATION

Let farms be harvested often, for example by removing or shortening the 28-day gate and adding a caller incentive like `StockBuyer`'s ramp, so that almost all of each lump lands in the cycle that earned it. Alternatively, allow one harvest per `StockPayouts` cycle, only inside a closing window such as its last day.

L-22 LOW ACKNOWLEDGED

The V3 add-liquidity bounds price exactly 1e18 raw units of token0 as an integer, so for cheap 18-decimal token0 pairs the bounds are coarse or divide by zero and deposits revert

AFFECTED CODE

- `src/robinhood/farm/libs/RHFarmKeeperV3Lib.sol`, _calculateTWAPBounds(), addLiquidity()
- `src/robinhood/oracles/V3TwapPriceProvider.sol`, rateX18()
- `src/robinhood/oracles/V4TwapPriceProvider.sol`, rateX18()

DESCRIPTION

`_calculateTWAPBounds` collapses the reference `sqrtPriceX96` into twapRatio, the integer number of raw token1 units for exactly 1e18 raw token0. It divides by that ratio at L317 and builds all four bounds from it. When 1e18 raw token0 is worth only a few raw token1 units, the ratio keeps one or two significant digits. An example is an 18-decimal token0 against 6-decimal USDG: at $0.00001 per token the ratio is 10, so the bounds can be off by up to 10%. This needs the cheap token to sort below the quote token. With the order reversed the ratio is huge and nothing is lost. When 1e18 raw token0 is worth less than one raw unit (below $0.000001 against USDG), the ratio is 0 and L317 panics with a division by zero. The V4 path and `calculateOptimalSwap` work on the full-precision sqrt price and do not have this loss. Both price providers' `rateX18` quote the same 1e18 raw units (V3, V4) in either address order, so swap floors out of such a token are equally coarse, and at a zero ratio both providers revert. validatePriceable() only checks `supportsPair`, so such a farm can be enabled.

IMPACT

At a zero ratio, every `depositToFarm` of the farm reverts. Just above zero, the truncated ratio sets mint minimums that the pool cannot meet whenever `liquiditySlippage` is below about half the rounding error (a true 10.9 truncated to 10 fails below about 4.3%). NFPM `mint` / `increaseLiquidity` then reverts, and so does the deposit. Above that threshold, each add still leaves part of the balance undeployed. Swaps out of the cheap token also accept up to the same error as extra slippage. The farm's ETH share and reinvested fees sit idle and earn nothing for stakers until the owner widens the slippage for every add, which does not help at a zero ratio, or removes the farm, which still works because removal prices its minimums at full precision. The current stock/USDG pairs ($10+ stocks) are not affected.

RECOMMENDATION

Quote the held amounts directly at full precision, as `calculateOptimalSwap` does, or size the add with `LiquidityAmounts` at the reference price, as the V4 library does, and treat a zero quote as an explicit error. Give the providers' `rateX18` the same precision, for example by raising the rate scale they share with `RHOracleSwapActions`.

L-23 LOW FIXED

Fixed liquidity cut leaves part of every V4 deposit unused, and the raised cap does not undo it

AFFECTED CODE

- `src/robinhood/farm/libs/RHFarmKeeperV4Lib.sol`, addLiquidity(), _prepareIncreaseUnlockDate()
- `src/robinhood/farm/RHFarmKeeper.sol`, depositToFarm()

DESCRIPTION

Every V4 deposit mints less liquidity than the farm's balances can fund. `_prepareIncreaseUnlockDate` sizes the position at the oracle price from the full balances, then cuts the liquidity by `liquiditySlippage`. The cut runs on every deposit, even when the pool price equals the oracle price. The tokens it holds back stay in `currency0Balance` and `currency1Balance` until a later `depositToFarm`.

The cut exists because the mint asks for an exact amount of liquidity, and the pool charges for it at its own price, which can differ from the oracle. The spare tokens cover that gap. The code never reads the pool price, so it cannot tell whether it needs the spare. `addLiquidity` receives `v4PoolManager` and the library attaches `StateLibrary`, but neither is used.

The code then raises each token cap by `liquiditySlippage`, to give the mint room above the planned amounts. The raise does not undo the cut, since `(1 - s) * (1 + s) = 1 - s²`. At 5% the cap stops at 99.75% of the balance, and at 25% it stops at 93.75%. A mint that needs more than the cap, but no more than the farm holds, reverts.

For example, take a full-range V4 farm with ETH as `currency0`. The pool and the oracle both price it at 1,000 TOKEN per ETH, and `liquiditySlippage` is 5%. After the rebalance swap, the farm holds 10 ETH and 10,000 TOKEN:

1. The add step sizes the position for 10 ETH and 10,000 TOKEN, then cuts it by 5%. The position now needs 9.5 ETH and 9,500 TOKEN.
2. It raises the caps by 5%, to 9.975 ETH and 9,975 TOKEN.
3. The mint takes 9.5 ETH and 9,500 TOKEN. 0.5 ETH and 500 TOKEN stay idle, though the farm could fund the full position at this price.
4. If the pool price instead sits 10.5% above the oracle, the same position needs 9,986 TOKEN. That is above the 9,975 cap but within the 10,000 the farm holds, so the mint reverts.

IMPACT

About `liquiditySlippage` of each deposit earns no LP fees until the next `depositToFarm`, which again holds back the same share. No funds are lost. The cap also reverts mints the farm could pay for, in a narrow band of pool prices: at the 5% setting, from 10.2% to 10.8% above the oracle, or from 9.3% to 9.75% below it.

RECOMMENDATION

Size the position at the pool's current price, read with `getSlot0` in the same call, after checking that this price is within `liquiditySlippage` of the oracle. Then drop the cut, and the existing cap at the balance bounds the mint. #24 and #25 suggest the same change, and it fixes this issue too.

If the add step keeps sizing at the oracle, cut with `Math.mulDiv(liquidity, BPS, BPS + farm.liquiditySlippage)` so the raise lands on the original amounts. Either use `v4PoolManager` or remove the parameter and the `StateLibrary` import.

L-24 LOW ACKNOWLEDGED

`validateV4Pool` accepts any hook, so a V4 farm pool's hook can keep up to `liquiditySlippage` of the farm's principal on adds and on removal, or lock the position by reverting on liquidity removal

AFFECTED CODE

- `src/robinhood/farm/libs/RHFarmKeeperLib.sol`, validateV4Pool()
- `src/robinhood/farm/libs/RHFarmKeeperV4Lib.sol`, _prepareDecreaseLiquidityUnlockData(), _prepareIncreaseUnlockDate(), collectFees()
- `src/robinhood/farm/RHFarmKeeper.sol`, enableV4Farm(), removeFarm()

DESCRIPTION

The fork deliberately removes L1's check that a V4 farm pool's hook is an OLY oracle, so that hookless stock pools can be farmed, and nothing replaces it. When the owner calls `enableV4Farm`, `validateV4Pool` checks the native-currency position and initialization, but it never inspects `poolKey.hooks`, its permission bits or an allow-list. The permission bits are encoded in the hook address and cannot change, so a dangerous permission is visible when the farm is enabled. However, nothing flags it to the owner, and an upgradeable hook's logic behind those bits can change. A hook that holds these permissions but is harmless when the farm is enabled can start taking deltas or reverting after an upgrade, without the keeper owner's involvement.

v4 runs the pool's `afterAddLiquidity` and `afterRemoveLiquidity` hooks on every keeper add and decrease, and a hook with the matching return-delta permission keeps part of the caller's delta. The v4 `PositionManager` bounds that only by the limits the keeper passes. It checks the principal against the decrease minimums, which the keeper sets `liquiditySlippage` below the reference amounts, and against the add maximums, which it sets `liquiditySlippage` above them. A fee collect is a zero-liquidity decrease, so the same check reverts on any take and the fees themselves cannot be kept. Instead, a hook that takes there, or any remove-liquidity hook that simply reverts, makes every collect revert. `depositToFarm`, both harvest functions and `removeFarm` (L852) all collect fees first.

IMPACT

The hook's controller can keep up to `liquiditySlippage` of the farm's principal when the farm is removed, and about the same share of each add. `liquiditySlippage` may be set as high as 50%. Alternatively, the hook can lock the whole position: collects, deposits, harvests and `removeFarm` revert for as long as the hook does so, and while that farm is due, `harvestAllFarms` reverts for every farm. This requires the owner to enable a V4 farm on a pool whose hook has a liquidity-removal or add-return-delta permission, and that hook to be malicious or to become malicious through an upgrade. Since only the owner enables farms and sets the slippage, the severity is Low.

RECOMMENDATION

In `validateV4Pool`, use `Hooks.hasPermission` to reject hooks with any liquidity-removal permission (`BEFORE_REMOVE_LIQUIDITY`, `AFTER_REMOVE_LIQUIDITY` and its return-delta flag) or with `AFTER_ADD_LIQUIDITY_RETURNS_DELTA`. Alternatively, require the hook to be on an owner-maintained allow-list. Swap-only hooks such as `OLYTaxHook` stay farmable.

L-25 LOW FIXED

A liquidity add sized from a lagging TWAP lets a caller mint the farm's position at the pre-move price

AFFECTED CODE

- `src/robinhood/farm/RHFarmKeeper.sol`, depositToFarm(), _addV3Liquidity(), _addV4Liquidity()
- `src/robinhood/farm/libs/RHFarmKeeperV3Lib.sol`, addLiquidity(), _calculateTWAPBounds()
- `src/robinhood/farm/libs/RHFarmKeeperV4Lib.sol`, addLiquidity()
- `src/robinhood/oracles/V3TwapPriceProvider.sol`, sqrtPriceX96()

DESCRIPTION

`depositToFarm` sizes its liquidity add, and the min amounts that floor it, from the pair's 1-hour TWAP through `sqrtPriceX96`, not from live spot. That TWAP does not move within a transaction and lags a real market move for up to a full window. After a real move, a caller pushes the pool's spot to the stale pre-move price, calls `depositToFarm` so the farm mints a full-range position at that stale ratio, and restores spot. The position was entered off the market price, so it holds less value than the tokens that funded it, and the caller captures the difference.

This is separate from the rebalance swap. The add is priced from the stale TWAP on its own, so it bites even when the farm already holds both tokens and no swap runs, for example when a harvest reinvests fees. It also costs in either direction, since entering a position at the wrong price loses value whichever way price has moved.

The same stale floor is also a liveness problem: when spot sits more than `liquiditySlippage` from the TWAP, the min amounts make the add revert, so deposits are blocked until the TWAP catches up.

Example, a farm already holding balanced WETH and NVDA:

1. NVDA moves against ETH. Spot reflects it, but the 1-hour TWAP still reads the pre-move price `T`.
2. A caller pushes spot to `T` and calls `depositToFarm`.
3. The add mints a full-range position at the `T` ratio, because both the sizing and the floor read `T`.
4. The caller restores spot. The position sits off the market price and the caller keeps the difference.

IMPACT

While a real move still lags the TWAP, any caller can make the farm mint its position at the pre-move price rather than the live one. This opens an extraction path on the add itself, with no swap required, and the amount grows with the gap. The loss falls on protocol revenue, which funds L1 stakers through StockPayouts. Separately, the same stale floor can block deposits whenever spot sits beyond `liquiditySlippage` of the TWAP.

RECOMMENDATION

Size the add and its min amounts from live spot, or from a reference the caller cannot move within one transaction that still tracks the market within a block, such as a short TWAP of about 60 seconds. Do not price the add from the 1-hour TWAP.

L-26 LOW FIXED

One farm that cannot harvest blocks `harvestAllFarms` for every farm

AFFECTED CODE

- `src/robinhood/farm/RHFarmKeeper.sol`, harvestAllFarms(), harvestFarm()
- `src/robinhood/farm/libs/RHFarmKeeperLib.sol`, distributeRewards()

DESCRIPTION

`harvestAllFarms` loops over every due farm and calls `_harvestFarm` directly, with no try/catch (L496). Each harvest collects the farm's fees and sends the staker share to `StockPayouts` with hard `safeTransfer` calls (L489-L497). If any one farm's fee collection or transfer reverts, the whole batch reverts.

The bug is triggered when two or more farms are due and one of them cannot finish its harvest. For example, the issuer of its stock token or USDG pauses transfers or freezes the keeper or `StockPayouts`, or its v4 hook reverts on fee collection.

IMPACT

`harvestAllFarms` reverts for every farm for as long as the one farm keeps failing. The stakers' 68% fee share from all healthy farms reaches `StockPayouts` late and can land in a later payout cycle, so it is paid to a different set of stakers. The failing farm's own `harvestFarm` and `removeFarm` also revert, so its LP and fees stay stuck until the issuer lifts the pause.

RECOMMENDATION

Run each farm's harvest through an external self-call wrapped in try/catch, emit a failure event and continue with the next farm, as `syncPayouts` already does.

INFORMATIONAL SEVERITY · 10 FINDINGS

I-01 INFORMATIONAL FIXED

`quoteSubmissionFee` comment names a stale field and the quote has no basefee margin

AFFECTED CODE

- `src/StockRelay.sol`, quoteSubmissionFee()

DESCRIPTION

`quoteSubmissionFee` sizes the `credit` calldata for a batch of `n` users and asks the inbox for the submission fee at the current basefee:

```
function quoteSubmissionFee(uint256 n) external view returns (uint256) {
    // selector + (cycleId, totalWeight, relayBlock, 3 array offsets) + 3 x (length word + n)
    uint256 dataLength = 4 + 32 * (9 + 3 * n);
    return inbox.calculateRetryableSubmissionFee(dataLength, block.basefee);
}
```

The comment lists `relayBlock` as the third head word, but `credit` takes `destEpoch` in that position. The formula itself is correct. The head holds three static words and three array offsets, and each of the three arrays adds a length word and `n` elements, for `9 + 3n` words. It matches the length of real `credit` calldata for `n` of 1, 2 and 5.

The quote also reads `block.basefee`, which has two weaknesses:

- Under `eth_call` with no gas price, geth-based nodes set `block.basefee` to zero, so an off-chain caller may get a quote of zero.
- The inbox checks `maxSubmissionCost` against the basefee of the block that includes `relay`. Basefee can rise by 12.5% per block, so a quote passed through unchanged can fall short.

IMPACT

The comment misleads anyone who checks the calldata layout against `credit`. A relayer who passes the quote straight into `relay` as `maxSubmissionCost` can have the call revert on L1 and lose the gas spent.

RECOMMENDATION

Replace `relayBlock` with `destEpoch` in the comment:

```
// selector + (cycleId, totalWeight, destEpoch, 3 array offsets) + 3 x (length word + n)
```

Document that callers must add a margin to the quote before they pass it as `maxSubmissionCost`, and that off-chain callers must set a gas price on the call. Alternatively, take the basefee as a parameter so the caller supplies it.

I-02 INFORMATIONAL FIXED

Removing the last USDG-quoted stock strands the leftover USDG

AFFECTED CODE

- `src/robinhood/stock/StockBuyer.sol` , _runBuys(), removeAsset(), sweepForeign()

DESCRIPTION

`buy` swaps ETH into USDG, then USDG into each USDG-quoted stock. Both steps run only when the basket holds at least one USDG-quoted stock:

```
if (usdgAssets != 0) {
    // ...
    perUsdg = IERC20(usdg).balanceOf(address(this)) / usdgAssets;
}
```

A round often leaves USDG behind:

- a USDG-quoted stock's swap fails (paused feed, thin pool, or a caller who moves the pool price) while the ETH to USDG swap succeeds.
- the retry ladder fills only part of a stock's share.
- `perUsdg` rounds down.

The code comment says this leftover "simply funds the next buy". That holds only while the basket holds a USDG-quoted stock. Once the owner removes the last one, for example after its feed dies, `usdgAssets` is zero in every later round and `buy` never touches USDG again. `sweepForeign` reverts on USDG, and nothing else can move it.

IMPACT

The leftover USDG buys no stock, and stakers never receive its value. The owner can get it back only by listing a USDG-quoted stock again. If the owner renounces ownership, the USDG stays locked for good.

RECOMMENDATION

In `removeAsset` , when the removed stock is the last USDG-quoted one, swap the contract's USDG back to ETH so the next `buy` spends it. Floor the swap with the oracle price, as `buyUsdg` does. `_swapExactIn` cannot pay out native ETH on a v3 venue, so on v3, swap to WETH and unwrap it.

The swap should not block the removal, because the owner needs to delist stocks whose feeds have died. If the swap fails, `buy` should retry it on each round while `usdgAssets` is zero and the USDG balance is not.

I-03 INFORMATIONAL FIXED

`RHTaxRouter` does not disable initializers on its implementation, so anyone can initialize it and become its owner

AFFECTED CODE

- `src/robinhood/stock/RHTaxRouter.sol`, initialize()
- `script/robinhood/DeployRobinhoodCore.s.sol`, _deployTaxRouter()

DESCRIPTION

`RHTaxRouter` has no constructor, so its `initialize()` is still callable on the implementation contract itself. The deploy script creates the implementation and initializes only the proxy, so anyone can later call `initialize` on the implementation and become its owner. The sibling `RHFarmKeeper` prevents this by calling `_disableInitializers()` in its constructor.

IMPACT

No funds are at risk. OpenZeppelin v5 restricts `upgradeToAndCall` to proxy calls, so the attacker cannot upgrade or destroy the implementation, and the proxy's state and ETH are untouched. The attacker only gains a contract that looks like the real router, which can mislead monitoring or integrators, and can redirect any ETH sent to the implementation address by mistake.

RECOMMENDATION

Add a constructor to `RHTaxRouter` that calls `_disableInitializers()`, as `RHFarmKeeper` does.

I-04 INFORMATIONAL FIXED

Nested swaps clear the swap flag while the outer swap is still running

AFFECTED CODE

- `src/robinhood/RHOracleSwapActions.sol` , _swapExactIn(), _isSwapping()
- `src/robinhood/farm/RHFarmKeeper.sol` , receive()

DESCRIPTION

`_swapExactIn` in `RHOracleSwapActions` sets a transient `SWAPPING_SLOT` flag before the retry ladder and clears it after:

```
SWAPPING_SLOT.asBoolean().tstore(true);
for (uint8 i; i < attempts; ++i) {
    // ...
    try this.executeSwap(v, tokenIn, tokenOut, attemptAmount, minOut) returns (uint256 out,
uint256 spent) {
        (amountOut, amountSpent) = (out, spent);
        break;
    } catch {
        // ...
    }
}
SWAPPING_SLOT.asBoolean().tstore(false);
```

Children read the flag through `_isSwapping` to tell swap proceeds apart from revenue. `RHFarmKeeper.receive` returns early while the flag is set, and otherwise passes `msg.value` to `distributeEth` as farm revenue.

`_swapExactIn` never checks the flag on entry, and on exit it writes a fixed `false` instead of restoring the earlier value. If a second `_swapExactIn` runs inside a first, for example a v3 swap started from a callback during a v4 swap, the inner call clears the flag when it returns. The rest of the outer swap then runs with `_isSwapping` returning `false` , so ETH it receives (v4 native output, v3 `refundETH`) reaches `receive` as revenue.

No current path can nest two swaps. Every entry point that reaches `_swapExactIn` (`RHFarmKeeper.depositToFarm` , `RHFarmKeeper.swapNativeForFarm` , `StockBuyer.buy` , `RHBuyNBurn.buyNBurn`) runs under a `nonReentrant` guard, and swaps within one call run one after another. The guard lives in each child, not in the base, so a new child or entry point that omits it inherits the defect.

IMPACT

None today. A future child or entry point that allows a nested swap would count the outer swap's ETH twice in `RHFarmKeeper`: once as swap output, which the library measures as a balance change, and once as farm revenue in `pendingEth`.

RECOMMENDATION

Enforce the no-nesting assumption in the base instead of relying on each child's guard. Either revert on entry when the flag is already set, or save the earlier value and restore it on exit:

```
bool wasSwapping = _isSwapping();
SWAPPING_SLOT.asBoolean().tstore(true);
// ... retry ladder ...
SWAPPING_SLOT.asBoolean().tstore(wasSwapping);
```

I-05 INFORMATIONAL ACKNOWLEDGED

`setAllocPoint` accepts 0 although enabling a farm requires at least 50

AFFECTED CODE

- `src/robinhood/farm/RHFarmKeeper.sol`, setAllocPoint(), receive()
- `src/robinhood/farm/libs/RHFarmKeeperLib.sol`, distributeEth()

DESCRIPTION

`addFarm` requires `allocPoint >= 50`, but `setAllocPoint` accepts any `uint16`, including 0. When two or more farms exist and all of them have 0 points, `totalAllocPoints` is 0. `distributeEth` then divides by it at L682 for every farm except the last, so the keeper's `receive` panics. `RHTaxRouter.distribute` catches the failed push and books the keeper's share in pending. `flush` keeps failing in the same way.

IMPACT

While every weight is 0, the keeper's 68% of each distribution stays parked in the router instead of reaching farms and stakers, and plain ETH transfers to the keeper revert. Setting every farm to 0 is a natural way to pause allocations, so ordinary owner configuration reaches this state. No funds are lost, because they move once a non-zero weight is restored.

RECOMMENDATION

Enforce the same minimum of 50 in `setAllocPoint`, or keep the ETH in `pendingEth` instead of splitting it when `totalAllocPoints` is 0.

I-06 INFORMATIONAL FIXED

`setDestinations` applies a new split to tax that has already accrued, so the owner can redirect the whole undistributed balance

AFFECTED CODE

- `src/robinhood/stock/RHTaxRouter.sol` , setDestinations(), _setDestinations(), distribute(), setGenesis()

DESCRIPTION

`distribute` splits the whole undistributed balance using whatever `destinations` holds when it is called, not the split that was in force while the tax arrived. `setDestinations` replaces the set without distributing first, so a new split applies retroactively to all accrued ETH. `_setDestinations` only rejects the zero address and requires the bps to sum to 10,000, so `[{to: owner, bps: 10_000}]` is accepted. A `setDestinations` call followed by the permissionless `distribute` therefore pushes the full balance to the owner. `setGenesis` also lets the owner point the genesis cut (up to 20%) at itself.

The NatSpec presents this as a "no fund-extraction path" and says the owner "cannot withdraw". It does disclose that the owner can change destinations, but it does not say that this is equivalent to a withdrawal of everything not yet distributed. The design is also inconsistent: `StockBuyer` makes its payout target immutable because a settable target "would let the owner redirect staker revenue", while the router that funds it keeps that power.

IMPACT

A compromised or malicious owner key can take all undistributed and future tax, starving the keeper (and through it `StockPayouts` stakers) and the burner. Only genesis ETH that is already credited is out of reach. An honest reconfiguration is also affected: any ETH that accrued under the old split is paid out under the new one. Integrators and users relying on the NatSpec will underestimate the owner's control.

RECOMMENDATION

Settle the undistributed balance under the current split inside `setDestinations` before replacing the set, skipping it when the balance is below `minDistribute` so the setter does not revert. Correct the NatSpec to state that changing destinations redirects all undistributed funds, and consider putting `setDestinations` , `setGenesis` and upgrades behind a timelock.

I-07 INFORMATIONAL FIXED

Removing a V3 farm leaves its empty position NFT unburned in the keeper

AFFECTED CODE

- `src/robinhood/farm/libs/RHFarmKeeperV3Lib.sol`, decreaseLiquidity()
- `src/robinhood/farm/RHFarmKeeper.sol`, removeFarm()

DESCRIPTION

When the owner calls `removeFarm` on a V3 farm, it withdraws all liquidity through the V3 `decreaseLiquidity`. That function collects the tokens and sets `farm.lp.liquidity` to zero, but never burns the NFT. `removeFarm` then deletes the farm record, so the keeper keeps an empty NFT that no code uses again.

The V4 path handles this case. On a full withdrawal it burns the position and resets `farm.lp.tokenId`. The V3 path has no matching step.

Adding the same pool back does not reuse the NFT either. `addFarm` starts every farm with `tokenId: 0`, so the next deposit mints a new one.

Example:

1. The owner adds a V3 farm. The first deposit mints NFT `#100` to the keeper.
2. The owner calls `removeFarm`. All liquidity is withdrawn and collected, so `#100` holds nothing. The keeper still owns it, and the farm record is deleted.
3. The owner adds the same pool again. The new farm starts with `tokenId: 0`, so the next deposit mints `#101`.
4. `#100` stays in the keeper forever. No function reads or uses it again.

IMPACT

Each removed V3 farm leaves one dead NFT in the keeper. The position manager keeps its entry, and tools that list the keeper's positions show empty V3 positions next to live ones.

V3 `increaseLiquidity` does not check the caller, so anyone can still add liquidity to a dead NFT. The keeper owns it but has no function to withdraw from it, so that liquidity and the fees it earns stay locked forever. Only whoever adds the liquidity loses funds. The protocol loses none.

RECOMMENDATION

On a full withdrawal in the V3 `decreaseLiquidity`, burn the NFT after the collect and reset `farm.lp.tokenId` to zero, as the V4 path does.

Because anyone can add liquidity to the NFT, a third party can add a tiny amount just before `removeFarm` and make `burn` revert with "Not cleared". Burn only when the position manager reports zero liquidity for the token, or wrap the call in `try` / `catch`, so such a deposit cannot block `removeFarm`.

I-08 INFORMATIONAL FIXED

V4 amount caps round down while the pool rounds the cost up, so deposits revert when the slippage raise rounds to zero

AFFECTED CODE

- `src/robinhood/farm/libs/RHFarmKeeperV4Lib.sol`, _prepareIncreaseUnlockDate(), addLiquidity()
- `src/robinhood/farm/libs/RHFarmKeeperLib.sol`, validateLiquiditySlippage()
- `src/robinhood/farm/RHFarmKeeper.sol`, depositToFarm()

DESCRIPTION

A V4 deposit reverts whenever the `liquiditySlippage` raise on a side rounds to zero, even when the pool price equals the oracle price.

The add step sets `amount0Max` and `amount1Max` with `LiquidityAmounts.getAmountsForLiquidity`, which rounds down. The pool charges the same liquidity with the same formula rounded up (`SqrtPriceMath`), and the PositionManager reverts with `MaximumAmountExceeded` when the charge is above the cap. At the same price, the pool asks for 1 wei more than the quote on each side. The Uniswap v4 guide warns against treating the two as equal, and the v4 SDK computes mint amounts with round-up for this reason.

Only the `liquiditySlippage` raise covers that wei. The raise, `amount * (10_000 + s) / 10_000`, also rounds down, so it adds nothing when a side's quote is below `10_000 / s` wei. `validateLiquiditySlippage` allows `s` down to 1 bps, which puts the limit at 10,000 wei. Without the raise, every add of any size reverts.

IMPACT

Today, deposits revert only while a side's quote is under `10_000 / liquiditySlippage` wei: 200 wei at 0.5%, 20 wei at 5%, and 10,000 wei at the 1 bps minimum. That is dust for 18-decimal tokens, but grows for low-decimal tokens.

RECOMMENDATION

Compute the caps with the pool's own rounding, as the v4 SDK's `mintAmounts` does: `SqrtPriceMath.getAmount0Delta(sqrtPriceX96, sqrtPriceUpper, liquidity, true)` and `SqrtPriceMath.getAmount1Delta(sqrtPriceLower, sqrtPriceX96, liquidity, true)`. The raise then only has to cover price movement.

I-09 INFORMATIONAL FIXED

Intermediary pairs that trade only in hookless v4 pools are unpriceable, so farms cannot route ETH through them

AFFECTED CODE

- `src/robinhood/farm/libs/RHFarmKeeperLib.sol`, validateFarmSetup(), setIntermediaryConfiguration(), setFarmPriceProvider()
- `src/robinhood/farm/RHFarmKeeper.sol`, _farmSwap(), _swapV3EthForFarmToken(), _swapV4EthForFarmToken()
- `src/robinhood/oracles/V3TwapPriceProvider.sol`, supportsPair(), _twap()
- `src/robinhood/oracles/V4TwapPriceProvider.sol`, setRoute()

DESCRIPTION

The problem starts when the only deep ETH liquidity for a farm token sits in a hookless v4 pool. The keeper cannot use that pool as the intermediary.

Each farm has one `priceProvider`, and the keeper asks it to price both the farm pair and the intermediary pair. `_farmSwap` takes its floor from `priceProvider.rateX18(tokenIn, tokenOut)`, and the v4 hop passes `farm.priceProvider`. At enable, `validateFarmSetup` requires that same provider to price the intermediary pair.

Neither provider can price a pair that trades only in a hookless v4 pool. `V3TwapPriceProvider` reads a v3 pool per pair and reports no support when none is registered. `V4TwapPriceProvider` rejects any pool without a hook. A v4 intermediary therefore works only when a v3 pool for the same pair exists for the provider to read.

Example:

1. The owner wants a v3 farm on `X` / `Y`, priced by `V3TwapPriceProvider`. Deep `ETH` / `X` liquidity sits in a hookless v4 pool, and no v3 `WETH` / `X` pool exists.
2. The owner calls `enableV3Farm` with that v4 pool as `intermediaryPoolKey`. `validatePriceable` finds no `WETH` / `X` route, and the call reverts with `FarmKeeper_UnpriceablePool`.
3. The owner tries to register the pool on `V4TwapPriceProvider`. `setRoute` reverts with `V4TwapPriceProvider__NotAnOracleHook`.
4. Moving the farm to a different provider does not help, because the one provider must price both `X` / `Y` and `ETH` / `X`. The farm cannot route its ETH through the v4 pool.

The setters also skip the check that enable runs. `setIntermediaryConfiguration` validates the new pool's tokens and existence but never calls `validatePriceable` on it, and `setFarmPriceProvider` checks only `farm.poolKey`. Either call can leave a live farm with an intermediary pair its provider cannot price. `depositToFarm` runs the hop first, `rateX18` reverts with `V3TwapPriceProvider__NoRoute`, and the farm takes no deposits until the owner undoes the change.

IMPACT

Farms cannot use v4 liquidity for the ETH hop unless the same pair also has a v3 pool. A farm token whose ETH liquidity sits only in a hookless v4 pool cannot be reached through that pool, and a farm with no such v3 pair for either token cannot be enabled as a non-native farm. Through the setters, one owner call can halt deposits on a live farm. No funds leave protocol control.

RECOMMENDATION

Let a farm price its intermediary pair apart from its farm pair. Either store a provider per intermediary pool, or add a routing provider that maps each pair to its own sub-provider. Add a provider that can price pairs found only in hookless v4 pools, for example a cross rate through two priced pairs, or document that such pairs cannot serve as intermediaries. Call `validatePriceable` on the new key in `setIntermediaryConfiguration`, and check the stored intermediary pair in `setFarmPriceProvider`.

I-10 INFORMATIONAL FIXED

A pause guard that returns any value other than 0 or 1 makes every quote on the route revert, instead of the documented "not paused"

AFFECTED CODE

- `src/robinhood/oracles/V3TwapPriceProvider.sol`, _requireFeedLive(), sqrtPriceX96(), rateX18()

DESCRIPTION

`_requireFeedLive` decodes the guard's `oraclePaused()` answer with `abi.decode(ret, (bool))` (L180). In Solidity 0.8 this decode reverts when the 32-byte word is neither 0 nor 1. The NatSpec (L173-L176) says malformed data is treated as "not paused" and that "only a clean `true` blocks a quote". In fact, every quote on the route reverts.

This happens when the route's `pauseGuard` returns a 32-byte value other than 0 or 1, such as a uint status code. That can be because the owner sets such a guard, or because the stock token issuer upgrades `oraclePaused()` to return one.

IMPACT

Every `sqrtPriceX96` (L80) and `rateX18` (L100) call for that route reverts. Every farm and StockBuyer path priced by the route is frozen until the owner replaces the guard with `setRoute`, including `removeFarm`, the farm's only exit.

RECOMMENDATION

Decode the result as `uint256` and treat only `== 1` as paused.